

IT3021
Data Warehousing & Business Intelligence

Assignment 1

Submitted By: Karunartahne U.M.T.N

IT23679726

Table of Contents

1. Data set selection.....	4
1.1. Data Set: NYC Taxi Trip Data	4
1.2. ER Diagram	5
2. Preparation of data sources.....	6
2.1. Trip Data	6
2.2. Location Data	7
2.3. Payment Type Data.....	8
2.4. Rate Code Data	9
2.5. Vendor Data	10
3. Solution Architecture	11
4. Data warehouse design & development.....	12
4.1. Assumptions.....	13
5. ETL Development	16
.....	17
5.1. Create the SSIS Package and Connection Managers.	17
5.2. Truncate StgTaxiTrip.....	18
5.3. Load StgTaxiTrip	19
5.4. Truncate StgTaxiZone.....	20
5.5. Load StgTaxiZone	21
5.6. Load DimDate.....	23
5.7. Load DimVendor.....	24
.....	24
5.8. Load DimPaymentType	25
.....	25
5.9. Load DimRateCode	27
.....	27
.....	28
5.10. Load DimLocation	29

5.11 . Update DimLocation Current Flag.....	32
5.12. Load FactTrip	33
.....	36
.....	37
.....	38
5.13. Update TripID	39
5.14. Execute the Package and Validate the Results	40
.....	41
6. ETL development – Accumulating fact tables	42
6.1. Extension of the Fact Table.....	42
6.2. Updating the Main ETL Package	42
6.3. Preparation of the Update Dataset.....	44
6.4. Creation of the Update Staging Table	44
6.5. Creation of the Separate SSIS Package	45

1. Data set selection

I have selected the New York City Taxi Trip dataset for the assignment.

1.1.Data Set: NYC Taxi Trip Data

<https://www.kaggle.com/datasets/neilclack/nyc-taxi-trip-data-google-public-data>

This dataset contains detailed taxi trip transactions records including vendor details, pickup and dropoff date and time, passenger count, trip distance, payment type, fare details, and pickup and dropoff location IDs. Each record in the dataset represents a single taxi trip. Therefore, this dataset can be considered as an OLTP dataset because it stores detailed transaction-level data and is not already organized into fact tables and dimension tables.

For this assignment, the main file selected from the dataset is taxi_trip_data.csv. This file contains the core trip transaction data required for data warehouse design. The dataset contains approximately one year of data for 2018. It also contains a large number of records and enough attributes to demonstrate multiple data sources, data warehousing designs, ETL tasks, hierarchies, dimensions, and aggregates.

The grain of the dataset is defined as: One record represents one taxi tip.

Some modifications were made in order to come up with a cleaner and more suitable dataset for the assignment.

The screenshot shows the Kaggle interface for the 'NYC Taxi Trip Data - Google Public Data' dataset. The left sidebar contains navigation links: Home, Competitions, Datasets (selected), Models, Benchmarks, Game Arena, Code, Discussions, Learn, and More. The main content area displays the dataset title, a subtitle 'A subset of the 2019 trip data in NYC Taxi Trip data available from Google.', and a thumbnail image of yellow taxis. Below the title are tabs for 'Data Card' (selected), 'Code (2)', 'Discussion (0)', and 'Suggestions (0)'. The 'About Dataset' section includes an 'About' subsection with text stating the data is a random subset of 10,000,000 rows from the Google BigQuery public datasets, and a list of files: 'taxi_trip_data.csv' and 'taxi_zone_geo.csv'. The 'Usability' section shows a score of 9.12. The 'License' section indicates 'Other (specified in description)'. The 'Expected update frequency' is 'Never'. The 'Tags' section includes 'Travel', 'Economics', 'Bigquery', and 'North America'. The 'Goal' section states the dataset was extracted for experimentation with machine learning models for price prediction.

1.2.ER Diagram

- The Location entity is used twice in the relationship structure because the same location master data can act as both pickup and dropoff points.

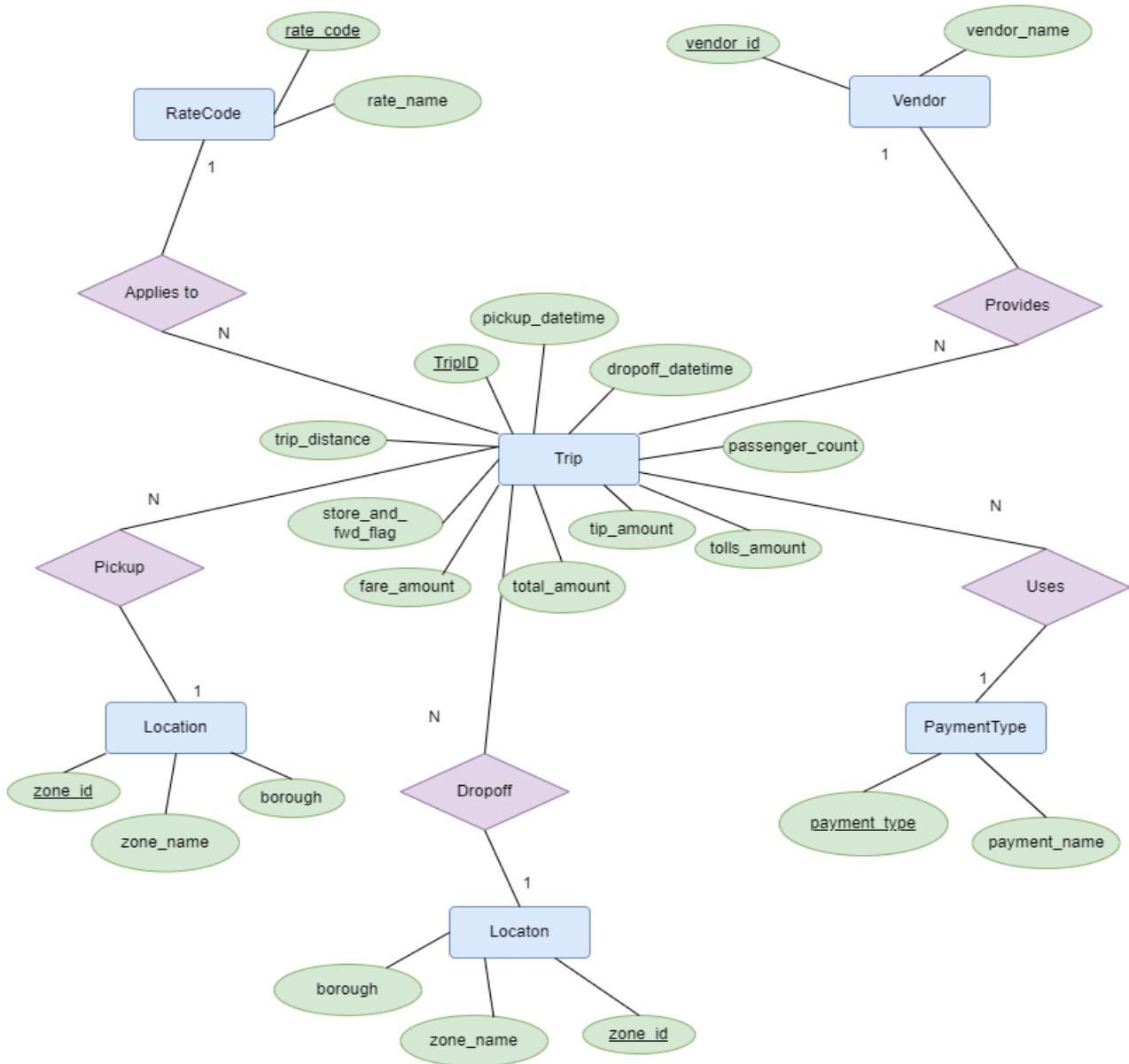


Figure 1 ER Diagram of the NYC Taxi Trip Operational Dataset

2. Preparation of data sources

The original Kaggle dataset contained three CSV files: taxi_trip_data.csv, taxi_zone_geo.csv, and original_cleaned_nyc_taxi_data_2018.csv. For this assignment, taxi_trip_data.csv was selected as the main transactional source. The file original_cleaned_nyc_taxi_data_2018.csv was not selected as a primary source because it already contains preprocessed and derived values. Modifications were done to the selected sources in order to create a more meaningful and manageable data extraction setup for the data warehouse.

As finale data sources, I have come up with Trip Data, Location Data, Payment Type Data, Rate Code Data, and Vendor Data.

2.1. Trip Data

Source type: CSV file

File name: taxi_trip_data.csv

This is the main transactional data source. It contains detailed taxi trip information such as vendor details, pickup and dropoff date and time, passenger count, trip distance, rate code, payment type, fare amount, tip amount, tolls amount, total amount, and pickup and dropoff location identifiers.

The column extra, mta_tax, and imp_surcharge were removed from the file because they are surcharge-related attributes with lower analytical importance compared to the main measures already retained. This source will mainly provide the data required for the central trip fact table.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	vendor_id	pickup_datetime	dropoff_datetime	passenger_count	trip_distance	rate_code	store_and_fwd_flag	payment_type	fare_amount	tip_amount	tolls_amount	total_amount	pickup_location_id	dropoff_location_id
2	2	3/29/2018 13:37	3/29/2018 14:17	1	18.15	3 N		1	70	16.16	10.5	96.96	161	1
3	2	3/29/2018 13:37	3/29/2018 14:15	1	4.59	1 N		1	25	5.16 0E-9		30.96	13	230
4	2	3/29/2018 13:26	3/29/2018 13:28	1	0.3	1 N		1	3	0.76 0E-9		4.56	231	231
5	2	3/29/2018 13:07	3/29/2018 14:03	2	16.97	1 N		1	49.5	5.61	5.76	61.67	231	138
6	2	3/29/2018 14:19	3/29/2018 15:19	5	14.45	1 N		1	45.5	10.41	5.76	62.47	87	138
7	2	3/29/2018 14:52	3/29/2018 15:51	1	11.6	1 N		1	42	14.57	5.76	63.13	68	138
8	1	3/29/2018 14:09	3/29/2018 14:36	1	5.8	1 N		1	24	4.95 0E-9		29.75	100	87
9	2	3/29/2018 15:21	3/29/2018 16:04	1	3.38	1 N		1	25	5.16 0E-9		30.96	144	161
10	2	3/29/2018 15:14	3/29/2018 16:42	1	16.98	3 N		1	85	15	12.5	112.8	87	1
11	2	3/29/2018 16:27	3/29/2018 16:58	1	4.99	1 N		1	22	4.76 0E-9		28.56	13	161
12	1	3/29/2018 16:27	3/29/2018 17:10	1	5.1	1 N		1	26.5	5.65 0E-9		33.95	186	33
13	1	3/29/2018 16:19	3/29/2018 16:53	2	8.7	1 N		1	28	7.1	5.76	42.66	138	237
14	2	3/29/2018 16:41	3/29/2018 17:56	1	11.11	1 N		1	45.5	10.61	5.76	63.67	163	138
15	2	3/29/2018 16:03	3/29/2018 17:05	1	9.54	1 N		1	41	9.71	5.76	58.27	138	244
16	2	3/29/2018 16:59	3/29/2018 17:51	1	9.18	1 N		1	34	7.48	5.76	49.04	161	138
17	1	3/29/2018 16:07	3/29/2018 17:31	1	9.8	1 N		1	50	17.25	5.76	74.81	140	138
18	1	3/29/2018 17:46	3/29/2018 18:38	1	5.8	1 N		1	32.5	4.5 0E-9		38.8	138	145
19	1	3/29/2018 17:17	3/29/2018 18:17	1	10	1 N		1	38.5	8.05 0E-9		48.35	138	25
20	1	3/29/2018 17:27	3/29/2018 18:21	1	9.8	1 N		1	38.5	6.94	5.76	53	138	90
21	1	3/29/2018 17:21	3/29/2018 17:54	1	5.4	1 N		1	24.5	5.25 0E-9		31.55	142	13
22	1	3/29/2018 17:07	3/29/2018 18:13	2	14.1	1 N		1	48.5	7	5.76	63.06	264	264

Figure 2. Trip Data.csv

2.2. Location Data

Source type: CSV file

File name: taxi_zone_geo.csv

This file is used as a secondary enrichment source. It contains descriptive information related to the location IDs used in the trip dataset. The attribute zone_geom was removed because it contains geographical polygon data. After cleaning, this source is used to enrich the pickup and dropoff location identifiers in the trip data.

	A	B	C	D	E	F	G
1	zone_id	zone_name	borough				
2		1 Newark Airport	EWB				
3		3 Allerton/Pelham Gardens	Bronx				
4		18 Bedford Park	Bronx				
5		20 Belmont	Bronx				
6		31 Bronx Park	Bronx				
7		32 Bronxdale	Bronx				
8		46 City Island	Bronx				
9		47 Claremont/Bathgate	Bronx				
10		51 Co-Op City	Bronx				
11		58 Country Club	Bronx				
12		59 Crotona Park	Bronx				
13		60 Crotona Park East	Bronx				
14		69 East Concourse/Concourse Village	Bronx				
15		78 East Tremont	Bronx				
16		81 Eastchester	Bronx				
17		94 Fordham South	Bronx				
18		119 Highbridge	Bronx				
19		126 Hunts Point	Bronx				
20		136 Kingsbridge Heights	Bronx				
21		147 Longwood	Bronx				
22		159 Melrose South	Bronx				
23		167 Morrisania/Melrose	Bronx				
24		168 Mott Haven/Port Morris	Bronx				

Figure3.taxi_zone_geo.csv

2.3.Payment Type Data

Table name: SrcPaymentType

Format: SQL Server table

A SQL Server lookup table was created to store payment type descriptions. In the trip file, payment information is stored as coded values. To improve readability and analytical usefulness, these codes are mapped to descriptive business names in a separate database source.

The screenshot displays a SQL Server query window with two tabs: 'SQLQuery3.s...tharu (72))*' and 'SQLQuery2.sq...l\tharu (56))*'. The active tab shows the following SQL script:

```
1 CREATE TABLE SrcPaymentType (  
2     PaymentID INT PRIMARY KEY,  
3     PaymentName VARCHAR(50) NOT NULL  
4 );  
5  
6 INSERT INTO SrcPaymentType (PaymentID, PaymentName)  
7 VALUES  
8 (1, 'Credit Card'),  
9 (2, 'Cash'),  
10 (3, 'No Charge'),  
11 (4, 'Dispute'),  
12 (5, 'Unknown'),  
13 (6, 'Voided Trip');  
14  
15 Select* from SrcPaymentType;
```

Below the script, the 'Results' tab is active, showing a table with 6 rows and 2 columns: 'PaymentID' and 'PaymentName'. The data is as follows:

	PaymentID	PaymentName
1	1	Credit Card
2	2	Cash
3	3	No Charge
4	4	Dispute
5	5	Unknown
6	6	Voided Trip

Figure 4. SrcPaymentType table

2.4. Rate Code Data

Table name: SrcRateCode

Format: SQL Server Table

A SQL Server lookup table was also created for rate codes. Similar to payment type, the trip file stores rate information as coded values, so a separate reference table improves data clarity and reporting.

The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows the SQL script for creating the SrcRateCode table and inserting data. The bottom pane shows the results of the query, displaying the table structure and data.

```
SQLQuery3.s...tharu (72))*  SQLQuery2.sq...l\tharu (56))*  
17 CREATE TABLE SrcRateCode (  
18     RateCodeID INT PRIMARY KEY,  
19     RateCodeName VARCHAR(100) NOT NULL  
20 );  
21  
22 INSERT INTO SrcRateCode (RateCodeID, RateCodeName)  
23 VALUES  
24 (1, 'Standard Rate'),  
25 (2, 'JFK'),  
26 (3, 'Newark'),  
27 (4, 'Nassau or Westchester'),  
28 (5, 'Negotiated Fare'),  
29 (6, 'Group Ride');  
30  
31 Select* from SrcRateCode;
```

100 % 3 0

Results Messages

	RateCodeID	RateCodeName
1	1	Standard Rate
2	2	JFK
3	3	Newark
4	4	Nassau or Westchester
5	5	Negotiated Fare
6	6	Group Ride

Figure 5. SrcRateCode table

2.5. Vendor Data

Table name: SrcVendor

Format: SQL Server table

A small vendor lookup table was created to convert vendor codes into descriptive vendor names. Although the source trip file already contains vendor_id, storing vendor descriptions in a separate table improves readability and produces a cleaner dimensional design.

The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows the SQL script for creating the SrcVendor table and inserting data. The bottom pane shows the results of the query, which is a table with two columns: VendorID and VendorName. The table contains two rows of data.

```
SQLQuery3.s...tharu (72))* X SQLQuery2.sq...l\tharu (56))*  
33 CREATE TABLE SrcVendor (  
34     VendorID INT PRIMARY KEY,  
35     VendorName VARCHAR(100) NOT NULL  
36 );  
37  
38 INSERT INTO SrcVendor(VendorID, VendorName)  
39 VALUES  
40 (1, 'Creative Mobile Technologies, LLC'),  
41 (2, 'VeriFone Inc.');
```

100 % No issues found

Results Messages

	VendorID	VendorName
1	1	Creative Mobile Technologies, LLC
2	2	VeriFone Inc.

Figure 6.SrcVendor table

3. Solution Architecture

A high-level DW and BI solution architecture was designed for the selected NYC Taxi Trip dataset to show how data flows from the source systems to the final reporting tools. This architecture includes the main components required for data extraction, transformation, storage, analysis, and reporting.

The solution architecture consists of the following components.

- Source data – This layer contains the prepared data sources identified in previous step. The sources are taxi_trip_data.csv, taxi_zone_geo.csv, and the SQL Server lookup tables payment_type, rate_code, and vendor.
- Staging database – This is a temporary storage area where data from the sources files and lookup tables is loaded before transformation. This layer is used to organize the extracted data and prepare it for the ETL process.
- ETL Layer – This layer is responsible for extracting data from the sources, transforming it into a suitable format, and loading it into the data warehouse.
- Data Warehouse – This is the central storage component of the solution. It stores the clean and transformed data in a dimensional format.
- OLAP layer – This layer is used for multidimensional analysis through measures, dimensions, and hierarchies.
- BI Layer – This is used by end users to analyze data and view business insights.

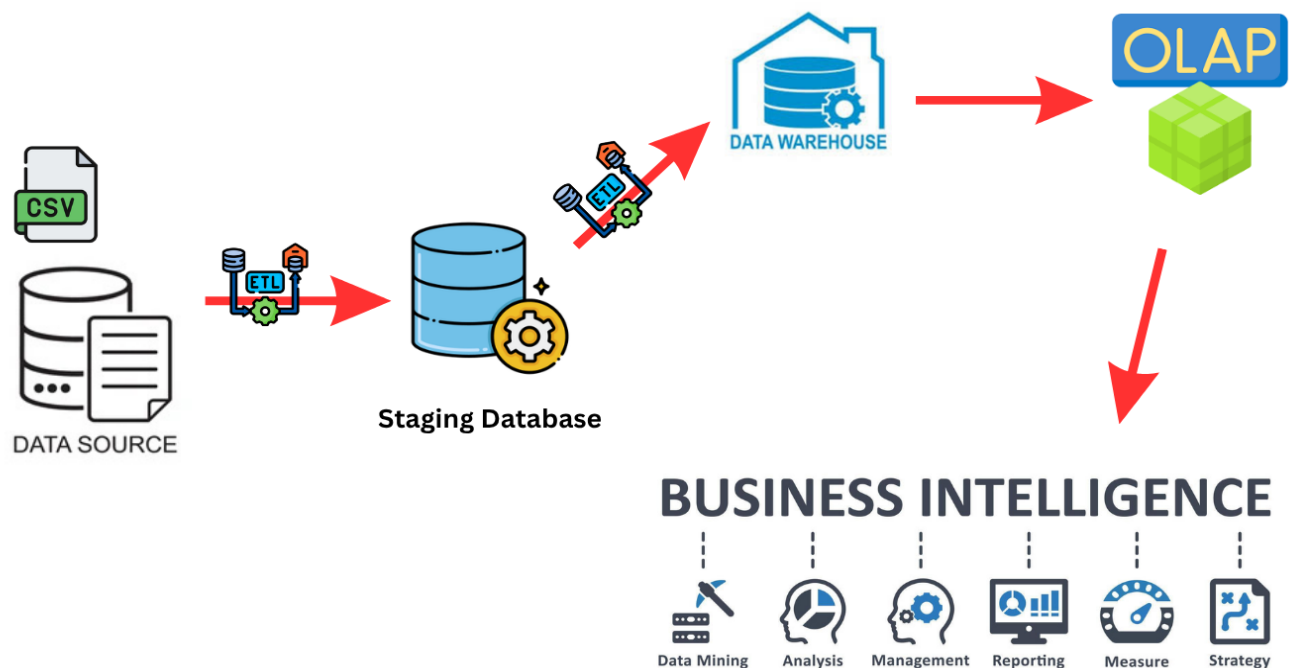


Figure7. Solution Architecture

4. Data warehouse design & development

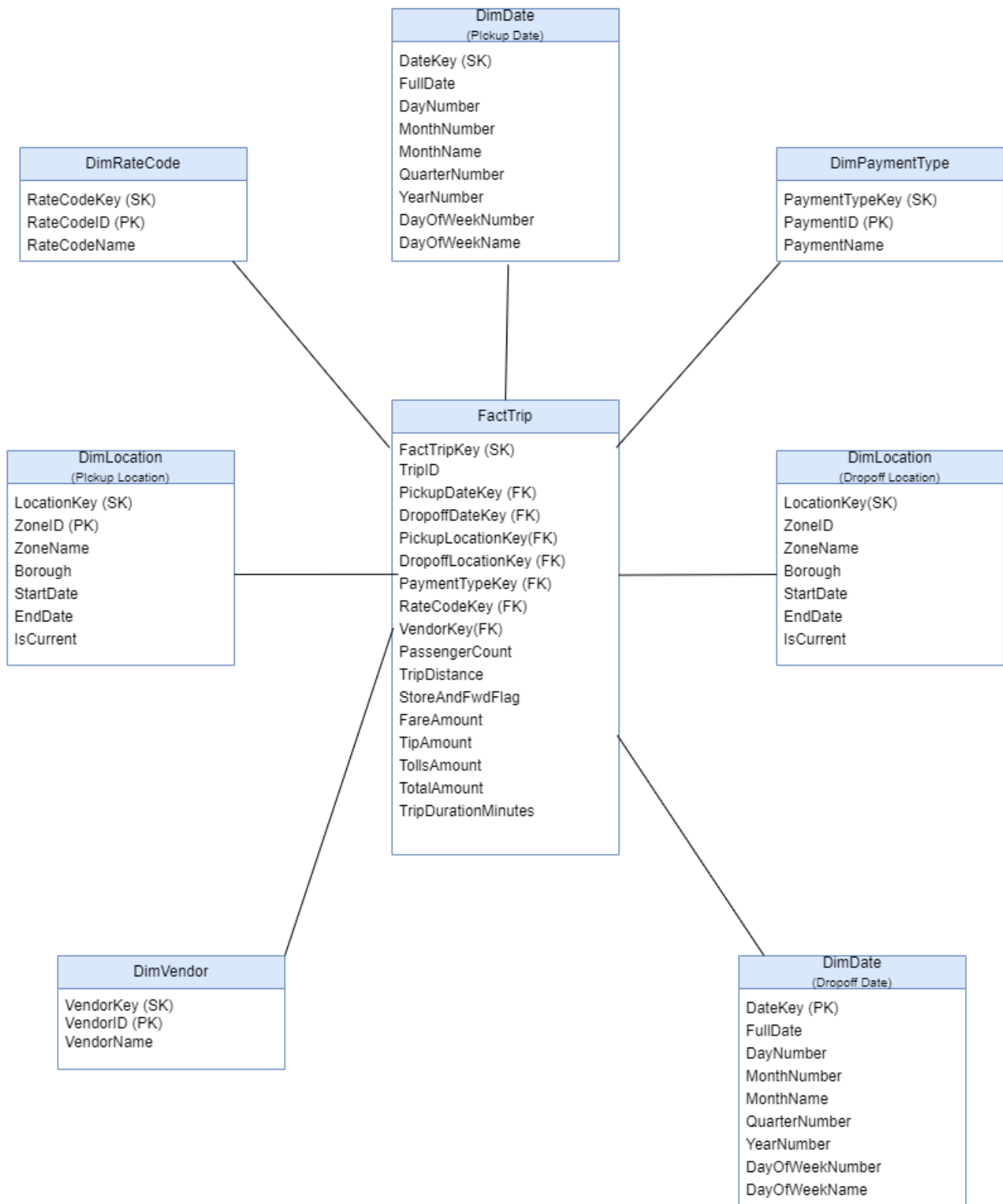


Figure 8. Star schema

4.1.Assumptions.

- A star schema is used for the data warehouse design. There is one fact table and five dimension tables
- One row in the fact table represents one taxi trip.
- The source dataset does not provide a single stable trip transaction key, therefore a generated identifier named TripID is used in the fact table.
- The same Date dimension is used twice in the fact table to represent pickup date and dropoff date.
- The same Location dimension is used twice in the fact table to represent pickup location and dropoff location.
- DimLocation is treated as a Slowly Changing Dimension.
- TripDurationMinutes is derived from the difference between dropoff_datetime and pickup_datetime.

```
SQLQuery2.s...tharu (56))* ✕
1  CREATE TABLE DimDate (
2      DateKey INT PRIMARY KEY,
3      FullDate DATE NOT NULL,
4      DayNumber INT NOT NULL,
5      MonthNumber INT NOT NULL,
6      MonthName VARCHAR(20) NOT NULL,
7      QuarterNumber INT NOT NULL,
8      YearNumber INT NOT NULL,
9      DayOfWeekNumber INT NOT NULL,
10     DayOfWeekName VARCHAR(20) NOT NULL
11 );
12
13 CREATE TABLE DimLocation (
14     LocationKey INT IDENTITY(1,1) PRIMARY KEY,
15     ZoneID INT NOT NULL,
16     ZoneName VARCHAR(100) NOT NULL,
17     Borough VARCHAR(50) NOT NULL,
18     StartDate DATE NOT NULL,
19     EndDate DATE NULL,
20     IsCurrent BIT NOT NULL
21 );
22
23 CREATE TABLE DimPaymentType (
24     PaymentTypeKey INT IDENTITY(1,1) PRIMARY KEY,
25     PaymentID INT NOT NULL,
26     PaymentName VARCHAR(50) NOT NULL
27 );
28
```

SQLQuery2.s...tharu (56))* ✨ ×

```
28
29 CREATE TABLE DimRateCode (
30     RateCodeKey INT IDENTITY(1,1) PRIMARY KEY,
31     RateCodeID INT NOT NULL,
32     RateCodeName VARCHAR(100) NOT NULL
33 );
34
35 CREATE TABLE DimVendor (
36     VendorKey INT IDENTITY(1,1) PRIMARY KEY,
37     VendorID INT NOT NULL,
38     VendorName VARCHAR(100) NOT NULL
39 );
40
```

SQLQuery2.s...tharu (56))* ✨ ×

```
40
41 CREATE TABLE FactTrip (
42     FactTripKey BIGINT IDENTITY(1,1) PRIMARY KEY,
43     TripID VARCHAR(100) NOT NULL,
44
45     PickupDateKey INT NOT NULL,
46     DropoffDateKey INT NOT NULL,
47     PickupLocationKey INT NOT NULL,
48     DropoffLocationKey INT NOT NULL,
49     PaymentTypeKey INT NOT NULL,
50     RateCodeKey INT NOT NULL,
51     VendorKey INT NOT NULL,
52
53     StoreAndFwdFlag CHAR(1) NULL,
54     PassengerCount INT NULL,
55     TripDistance DECIMAL(10,2) NULL,
56     FareAmount DECIMAL(10,2) NULL,
57     TipAmount DECIMAL(10,2) NULL,
58     TollsAmount DECIMAL(10,2) NULL,
59     TotalAmount DECIMAL(10,2) NULL,
60     TripDurationMinutes INT NULL
61 );
62
```

SQLQuery2.s...tharu (56))*

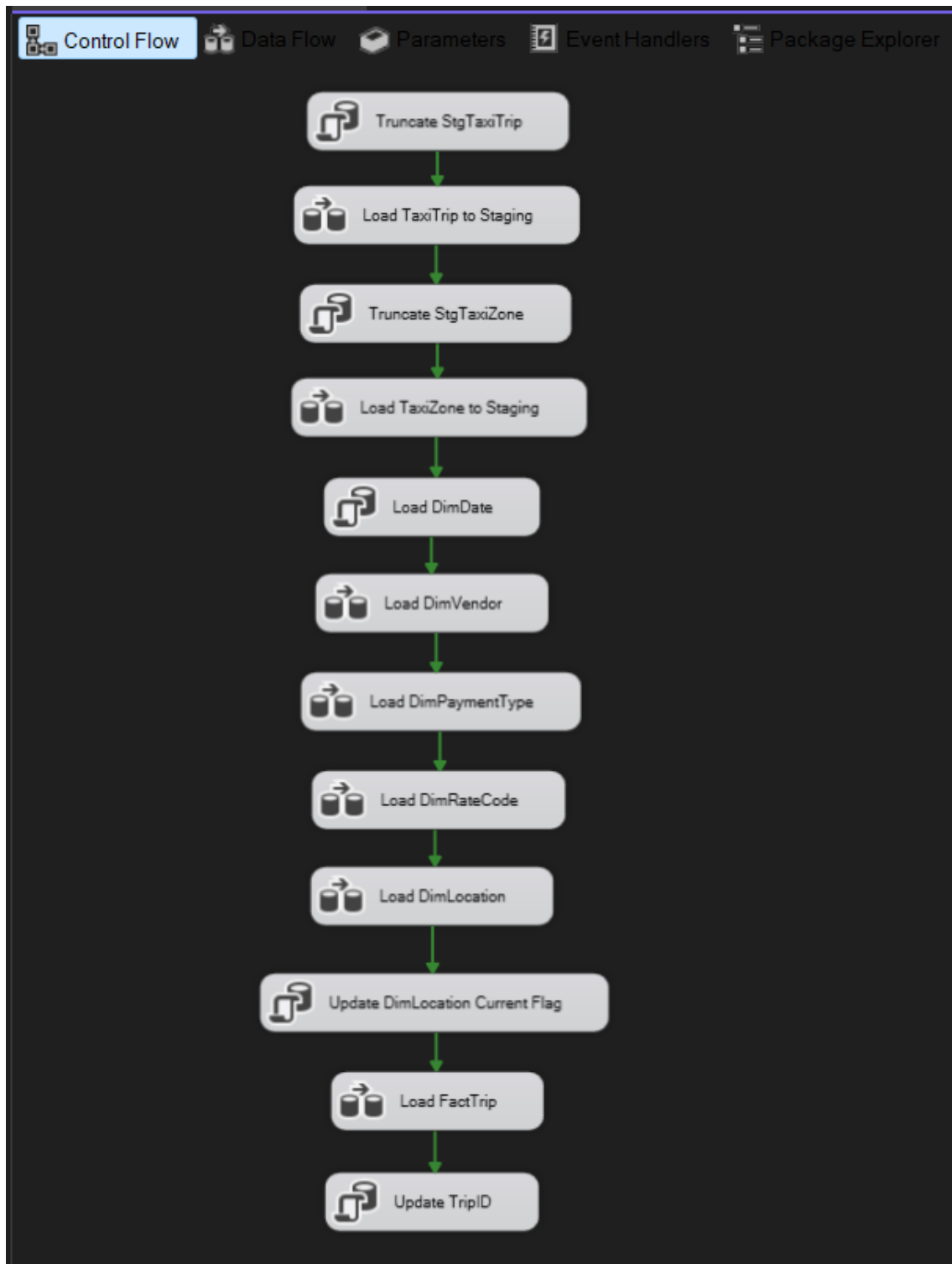
```
61 );
62
63 ALTER TABLE FactTrip
64 ADD CONSTRAINT FK_FactTrip_PickupDate
65 FOREIGN KEY (PickupDateKey) REFERENCES DimDate(DateKey);
66
67
68 ALTER TABLE FactTrip
69 ADD CONSTRAINT FK_FactTrip_DropoffDate
70 FOREIGN KEY (DropoffDateKey) REFERENCES DimDate(DateKey);
71
72
73 ALTER TABLE FactTrip
74 ADD CONSTRAINT FK_FactTrip_PickupLocation
75 FOREIGN KEY (PickupLocationKey) REFERENCES DimLocation(LocationKey);
76
77
78 ALTER TABLE FactTrip
79 ADD CONSTRAINT FK_FactTrip_DropoffLocation
80 FOREIGN KEY (DropoffLocationKey) REFERENCES DimLocation(LocationKey);
81
82
83 ALTER TABLE FactTrip
84 ADD CONSTRAINT FK_FactTrip_PaymentType
85 FOREIGN KEY (PaymentTypeKey) REFERENCES DimPaymentType(PaymentTypeKey);
86
87
88 ALTER TABLE FactTrip
89 ADD CONSTRAINT FK_FactTrip_RateCode
90 FOREIGN KEY (RateCodeKey) REFERENCES DimRateCode(RateCodeKey);
```

TAXI_TRIP_DATA_DW

- Database Diagrams
- Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo.DimDate
 - dbo.DimLocation
 - dbo.DimPaymentType
 - dbo.DimRateCode
 - dbo.DimVendor
 - dbo.FactTrip
 - Dropped Ledger Tables

Figure 9. implemented Dim and Fact tables

5. ETL Development



FigureError! No text of specified style in document..10. Control flow

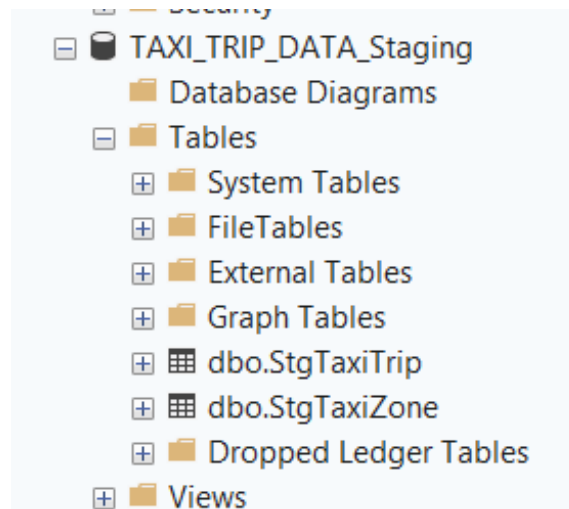


Figure.12. TAXI_TRIP_DATA_Staging

5.1.Create the SSIS Package and Connection Managers.

First, an SSIS package was created in Visual Studio. After that, connection managers were created for the source database, staging database, data warehouse database, and CSV source files.

The following connection managers were used:

- CM_SRC_IT23679726
- CM_STG_TAXI_TRIP_DATA_Staging
- CM_DW_TAXI_TRIP_DATA_DW
- CM_FF_TaxiTrip
- CM_FF_TaxiZone

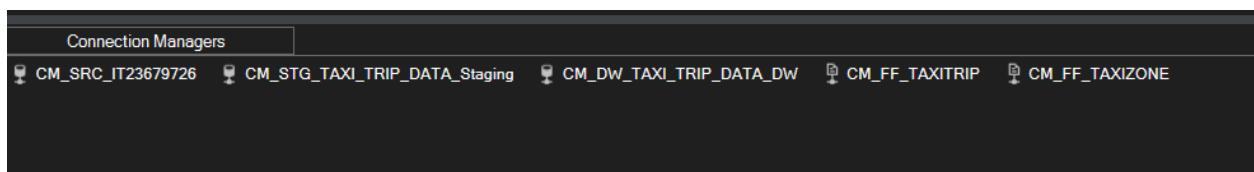


Figure 11. Connection Managers

5.2.Truncate StgTaxiTrip

An Execute SQL Task was used to truncate the StgTaxiTrip table before loading new trip records. This was done to avoid duplicate data when the package is executed multiple times.

Execute SQL Task Editor

Configure the properties required to run SQL statements and stored procedures using the selected connection.

General
Parameter Mapping
Result Set
Expressions

General	
Name	Truncate StgTaxiTrip
Description	Execute SQL Task
Options	
TimeOut	0
CodePage	1252
TypeConversionMode	Allowed
Result Set	
ResultSet	None
SQL Statement	
ConnectionType	OLE DB
Connection	CM_STG_TAXI_TRIP_DATA_Staging
SQLSourceType	Direct input
SQLStatement	TRUNCATE TABLE dbo.StgTaxiTrip;
IsQueryStoredProcedure	False
BypassPrepare	True

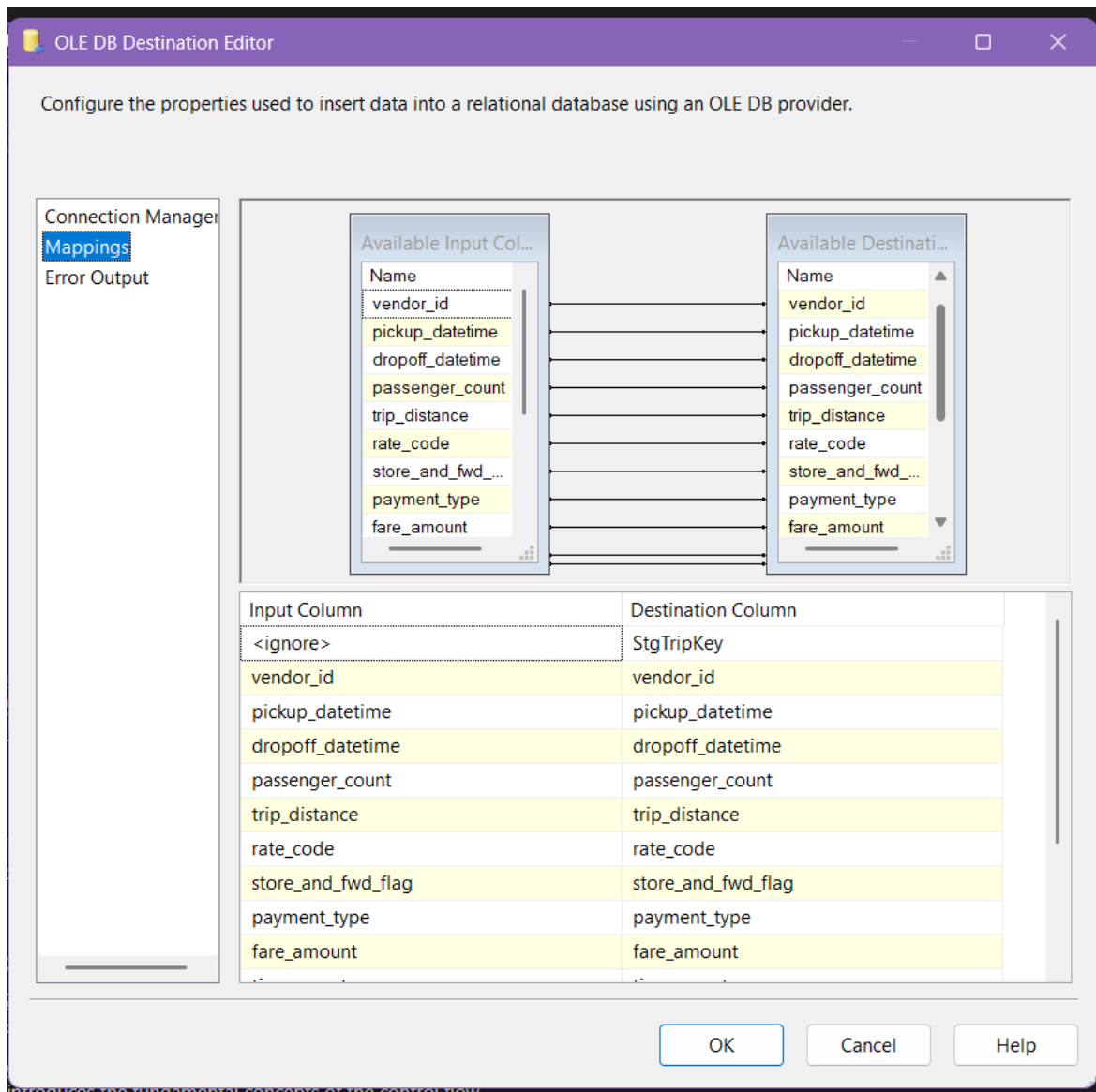
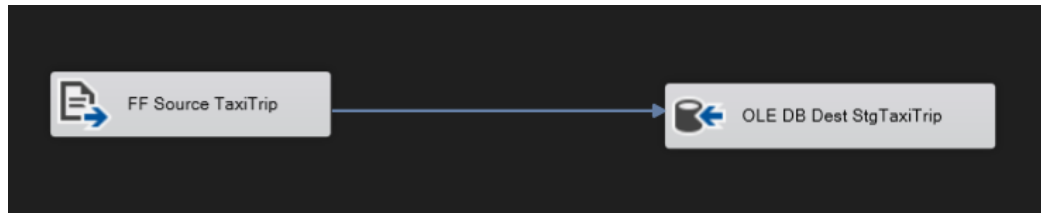
Name
Specifies the name of the task.

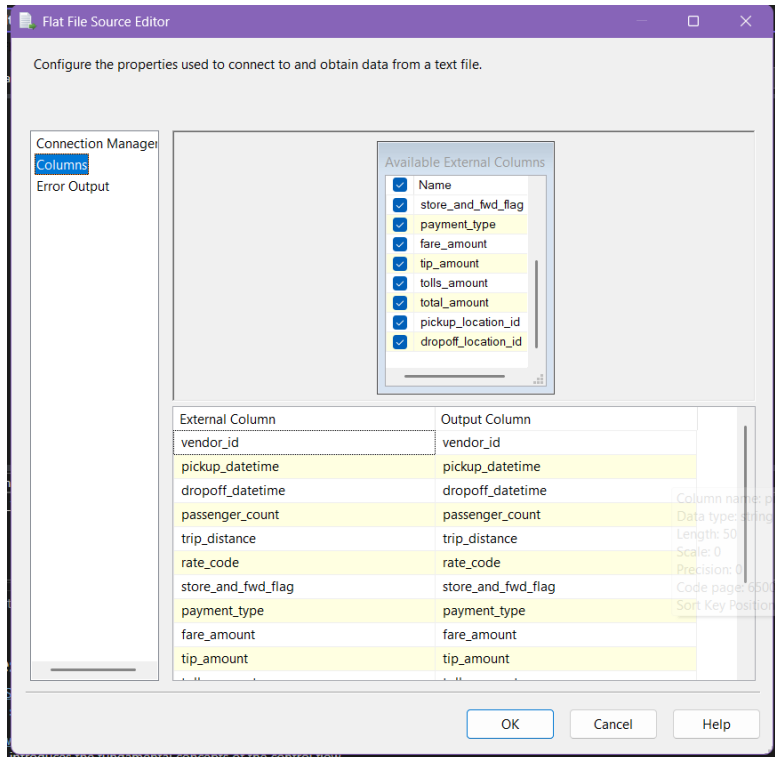
Browse... Build Query... Parse Query

OK Cancel Help

5.3.Load StgTaxiTrip

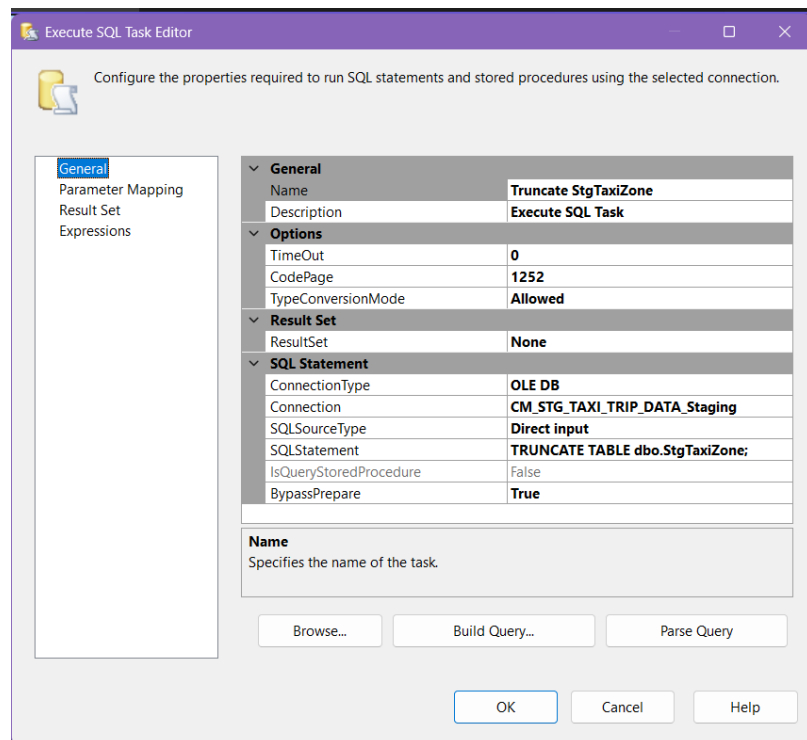
Trip data was extracted from the CSV source and loaded into StgTaxiTrip.





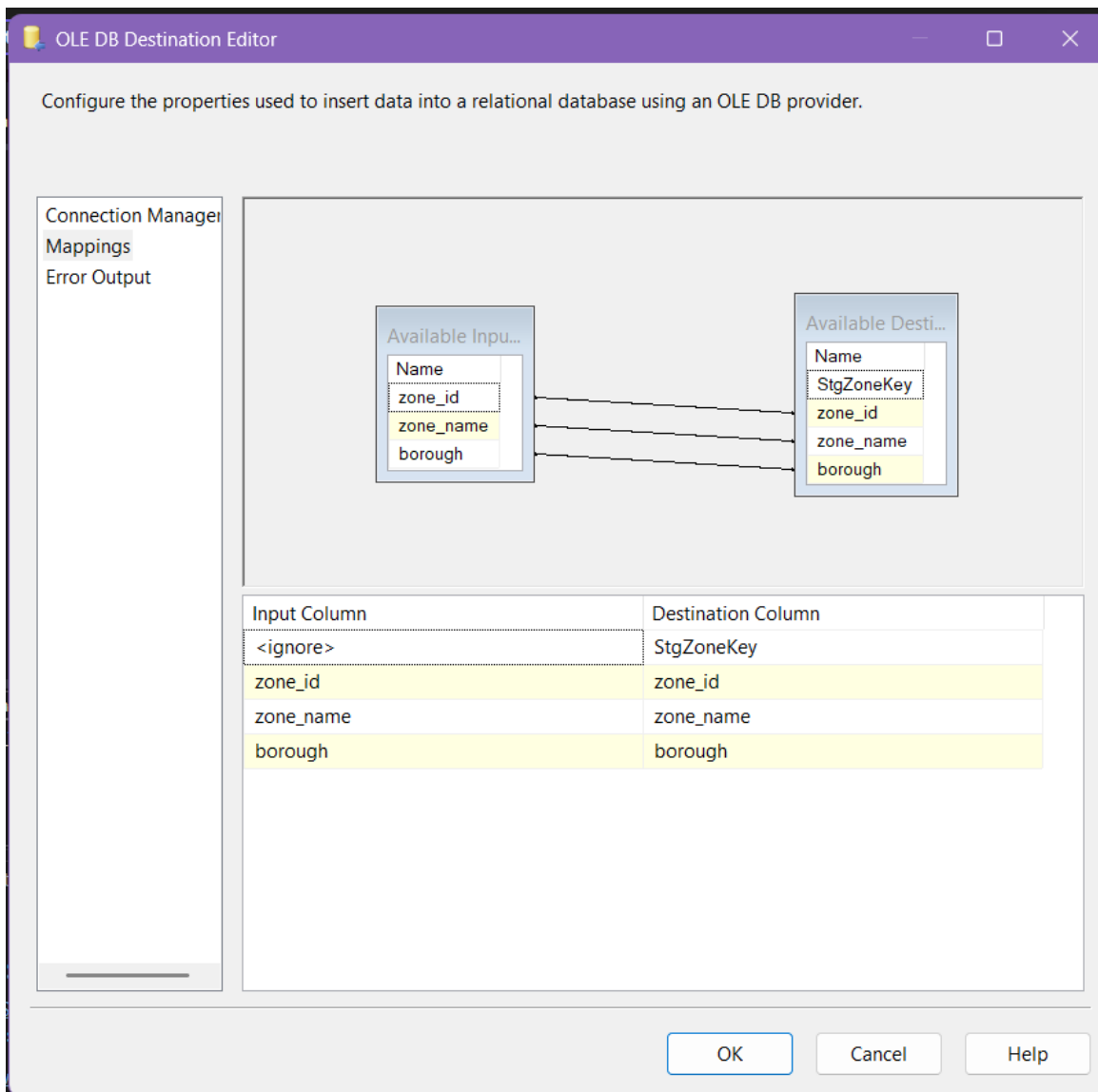
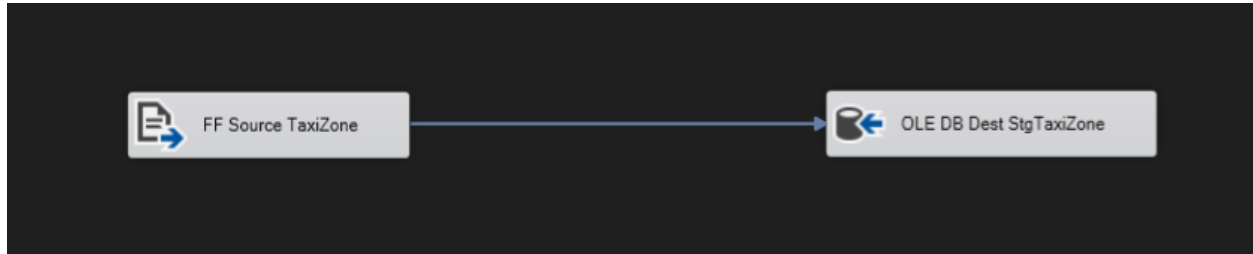
5.4.Truncate StgTaxiZone

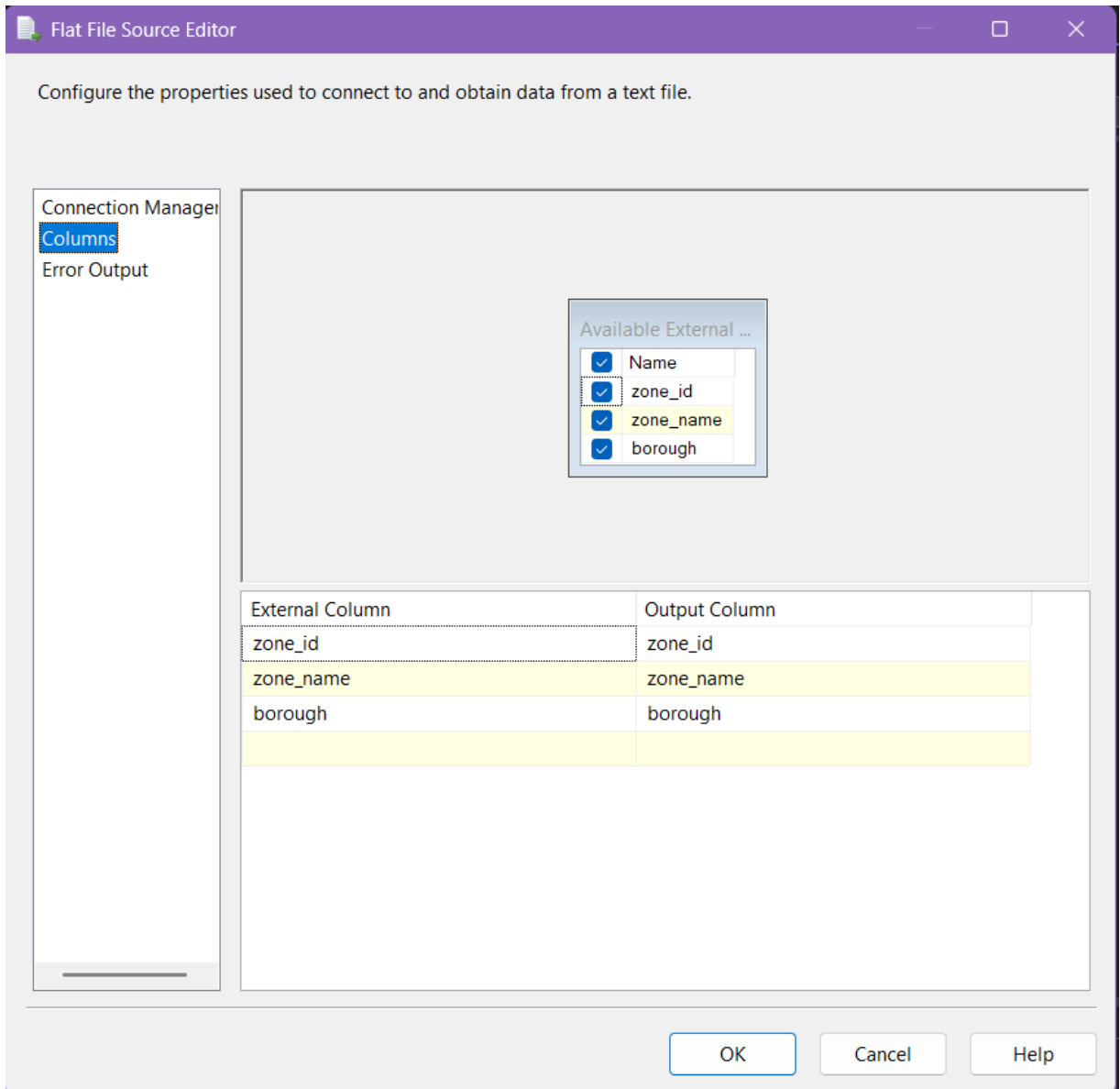
The StgTaxiZone staging table was cleared before loading new zone records.



5.5.Load StgTaxiZone

Zone data was extracted from the CSV source and loaded into StgTaxiZone.





5.6. Load DimDate

The date dimension was populated before loading the fact table.

Execute SQL Task Editor

Configure the properties required to run SQL statements and stored procedures using the selected connection.

General
Parameter Mapping
Result Set
Expressions

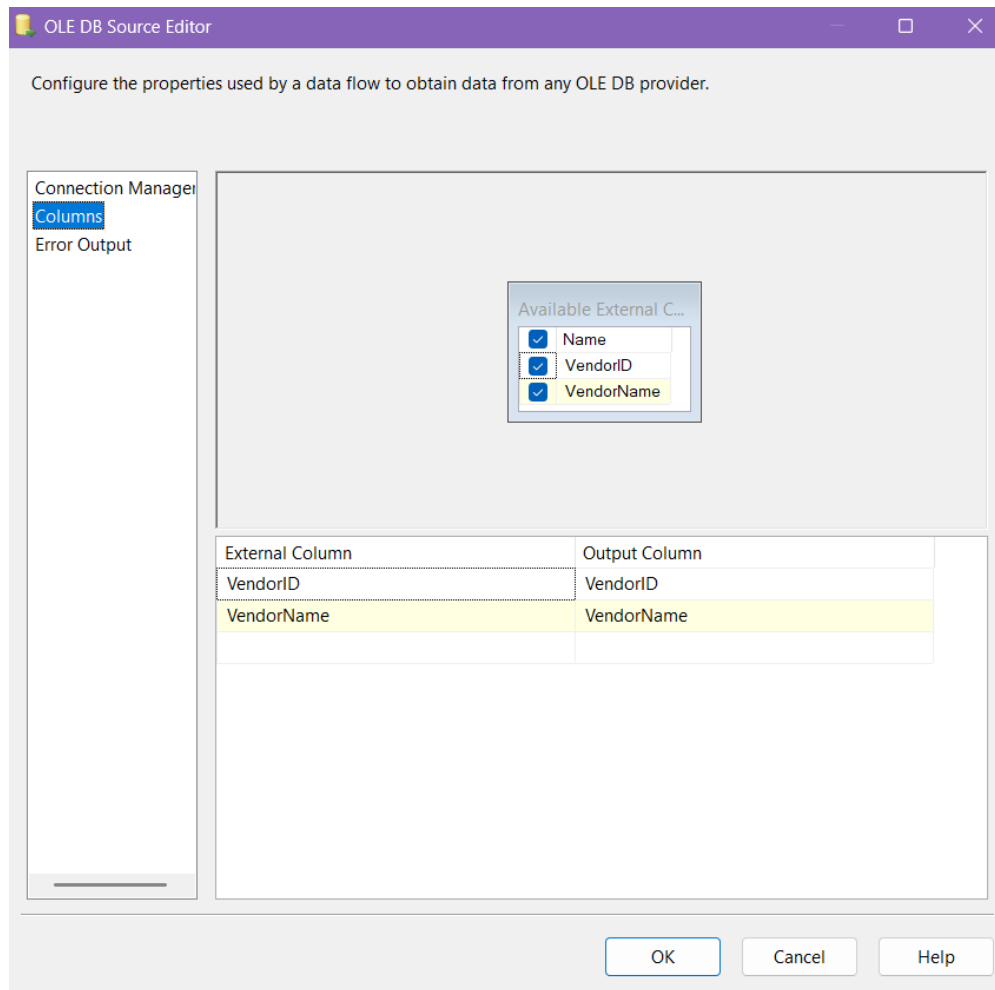
General	
Name	Load DimDate
Description	Execute SQL Task
Options	
TimeOut	0
CodePage	1252
TypeConversionMode	Allowed
Result Set	
ResultSet	None
SQL Statement	
ConnectionType	OLE DB
Connection	CM_DW_TAXI_TRIP_DATA_DW
SQLSourceType	Direct input
SQLStatement	DELETE FROM DimDate; WITH d AS (SELECT ...
IsQueryStoredProcedure	False
BypassPrepare	True

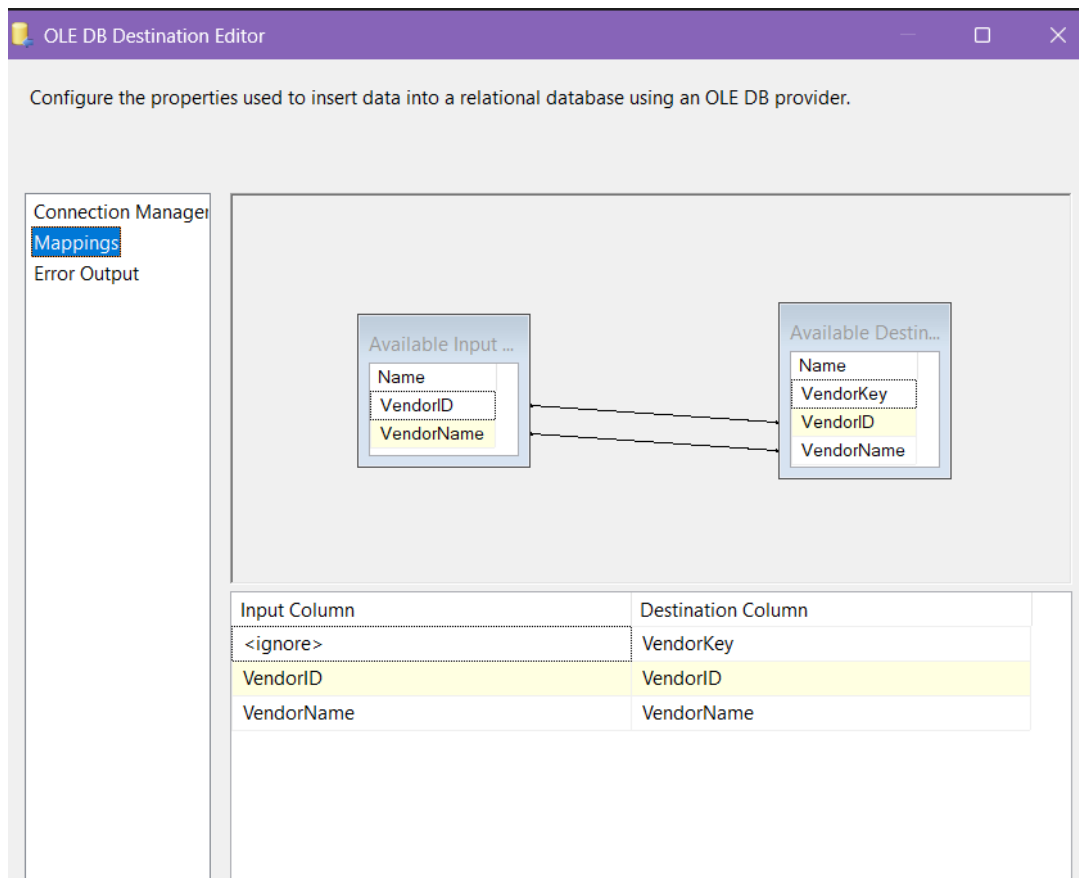
Name
Specifies the name of the task.

Browse... Build Query... Parse Query

OK Cancel Help

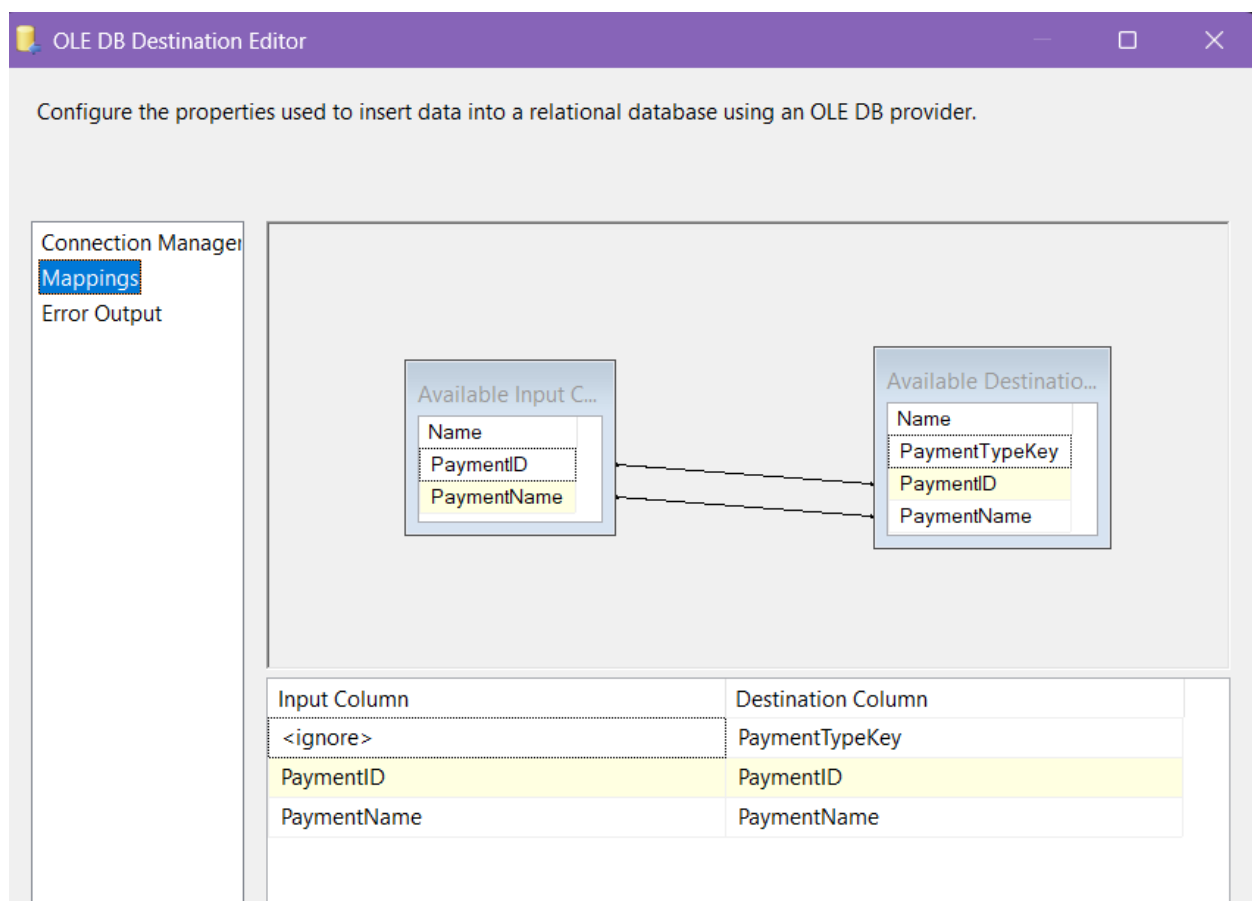
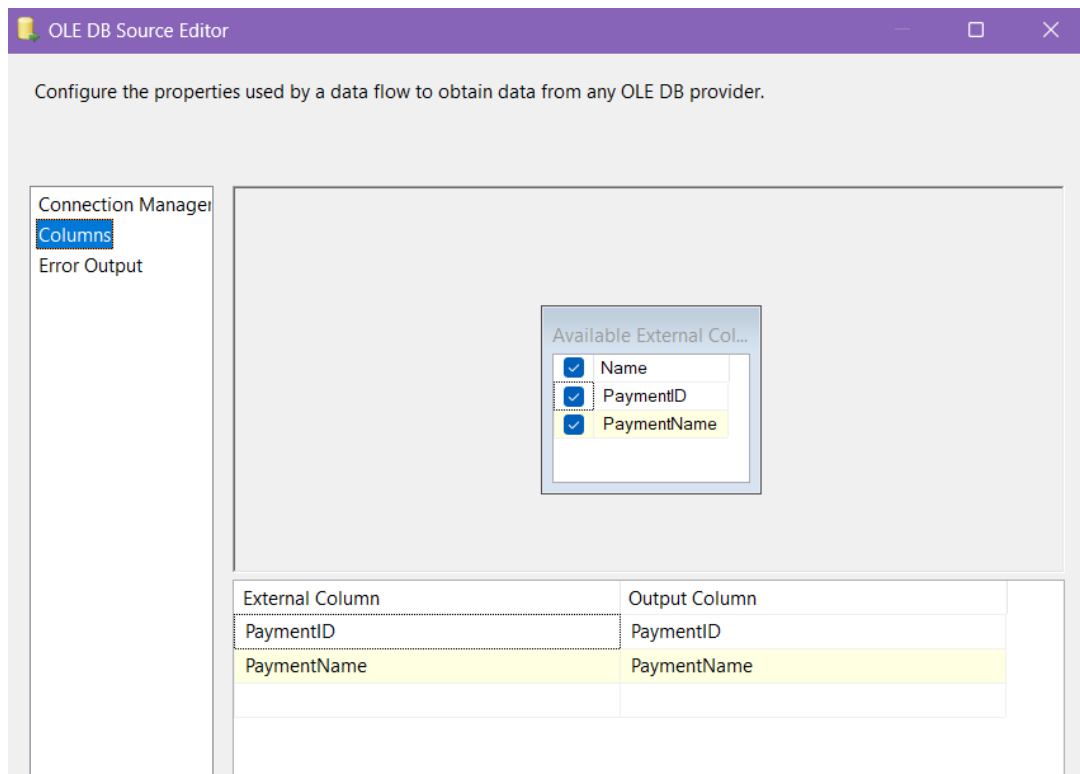
5.7. Load DimVendor





5.8. Load DimPaymentType





5.9. Load DimRateCode



OLE DB Source Editor

Configure the properties used by a data flow to obtain data from any OLE DB provider.

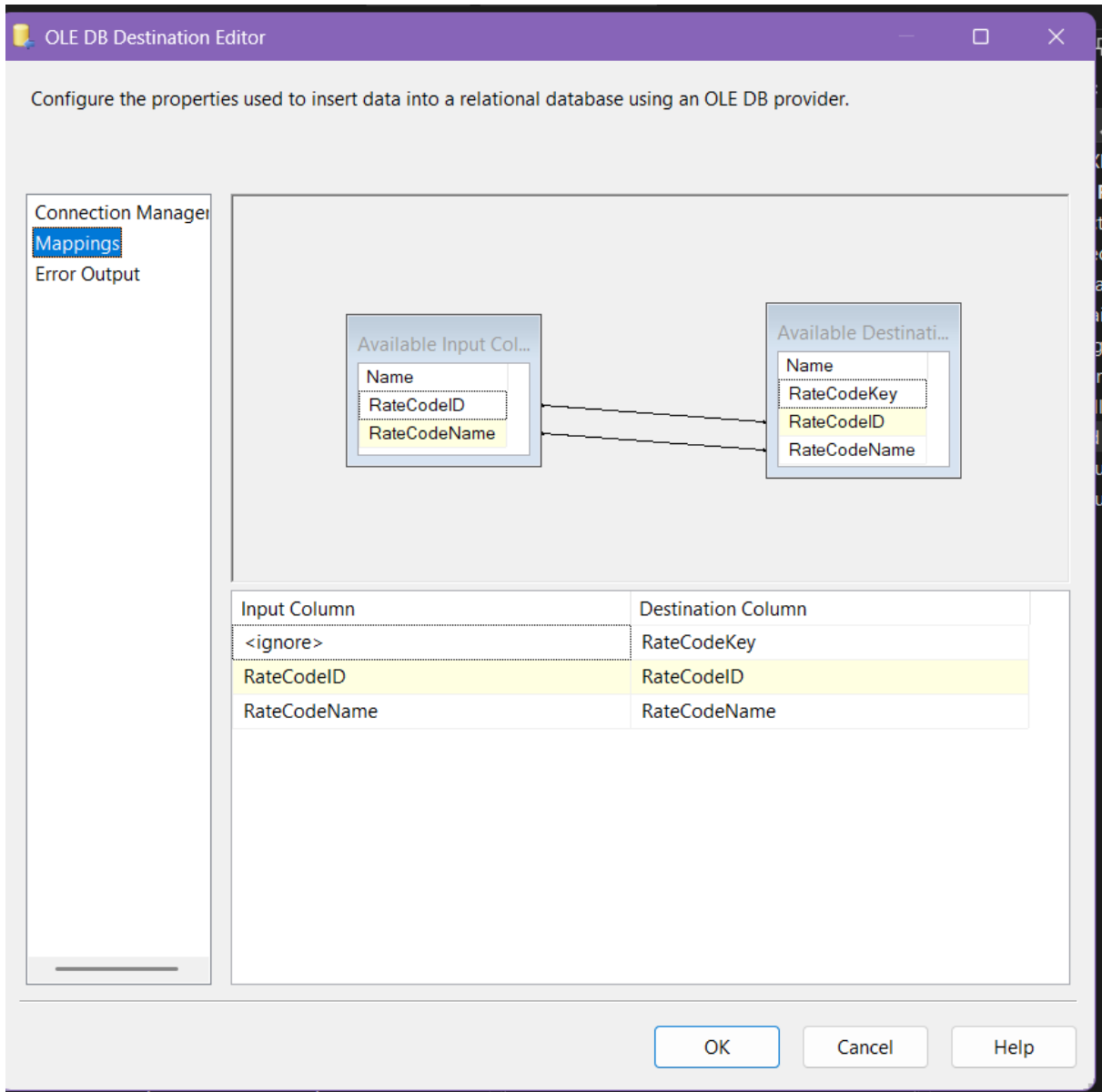
Connection Manager
Columns
Error Output

Available External Colu...

☒	Name
☒	RateCodeID
☒	RateCodeName

External Column	Output Column
RateCodeID	RateCodeID
RateCodeName	RateCodeName

OK Cancel Help

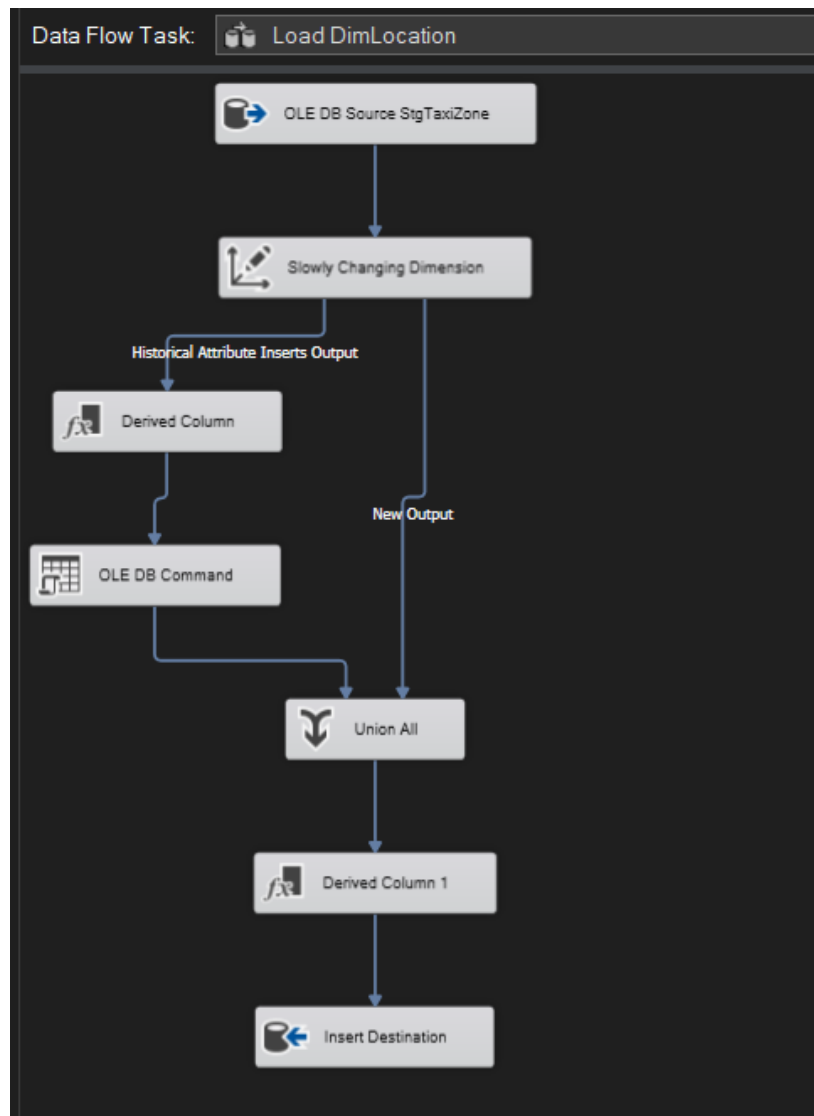


5.10. Load DimLocation

A Data Flow Task was used to load the location dimension from StgTaxiZone into DimLocation. Since DimLocation was selected as the Slowly Changing Dimension, the Slowly Changing Dimension transformation was used in this step.

In this step:

- StgTaxiZone was used as the source
- ZoneID was used as the business key
- ZoneName and Borough were treated as historical attributes
- new and changed records were loaded into DimLocation
- StartDate and EndDate were used to maintain history



Slowly Changing Dimension Wizard

Select a Dimension Table and Keys

Select a dimension table to load and map columns in the transformation input to columns in the

Connection manager:
 CM_DW_TAXI_TRIP_DATA_DW New...

Table or view:
 [dbo].[DimLocation]

Input Columns	Dimension Columns	Key Type
borough	Borough	Not a key column
	EndDate	
	IsCurrent	
	StartDate	
zone_id	ZoneID	Business key
zone_name	ZoneName	Not a key column

Help < Back Next > Finish >>| Cancel

Slowly Changing Dimension Wizard

Slowly Changing Dimension Columns

Manage the changes to column data in your slowly changing dimensions by setting the change

Fixed Attribute
 Select this type when the value in a column should not change. Changes are treated as errors.

Changing Attribute
 Select this type when changed values should overwrite existing values. This is a Type 1 change.

Historical Attribute
 Select this type when changes in column values are saved in new records. Previous values are saved in records marked as outdated. This is a Type 2 change.

Select a change type for slowly changing dimension columns:

Dimension Columns	Change Type
Borough	Historical attribute
ZoneName	Historical attribute

Help < Back Next > Finish >>| Cancel

Slowly Changing Dimension Wizard

Historical Attribute Options

You can record historical attributes using a single column or start and end date columns.

☐ Use a single column to show current and expired records

Column to indicate current record:

Value when current:

Expiration value:

☒ Use start and end dates to identify current and expired records

Start date column:

End date column:

Variable to set date values:

Help < Back Next > Finish >>| Cancel

Slowly Changing Dimension Wizard

Finish the Slowly Changing Dimension Wizard

Review the outputs that the Slow Changing Dimension Wizard will build to support a slowly

The following outputs will be configured:

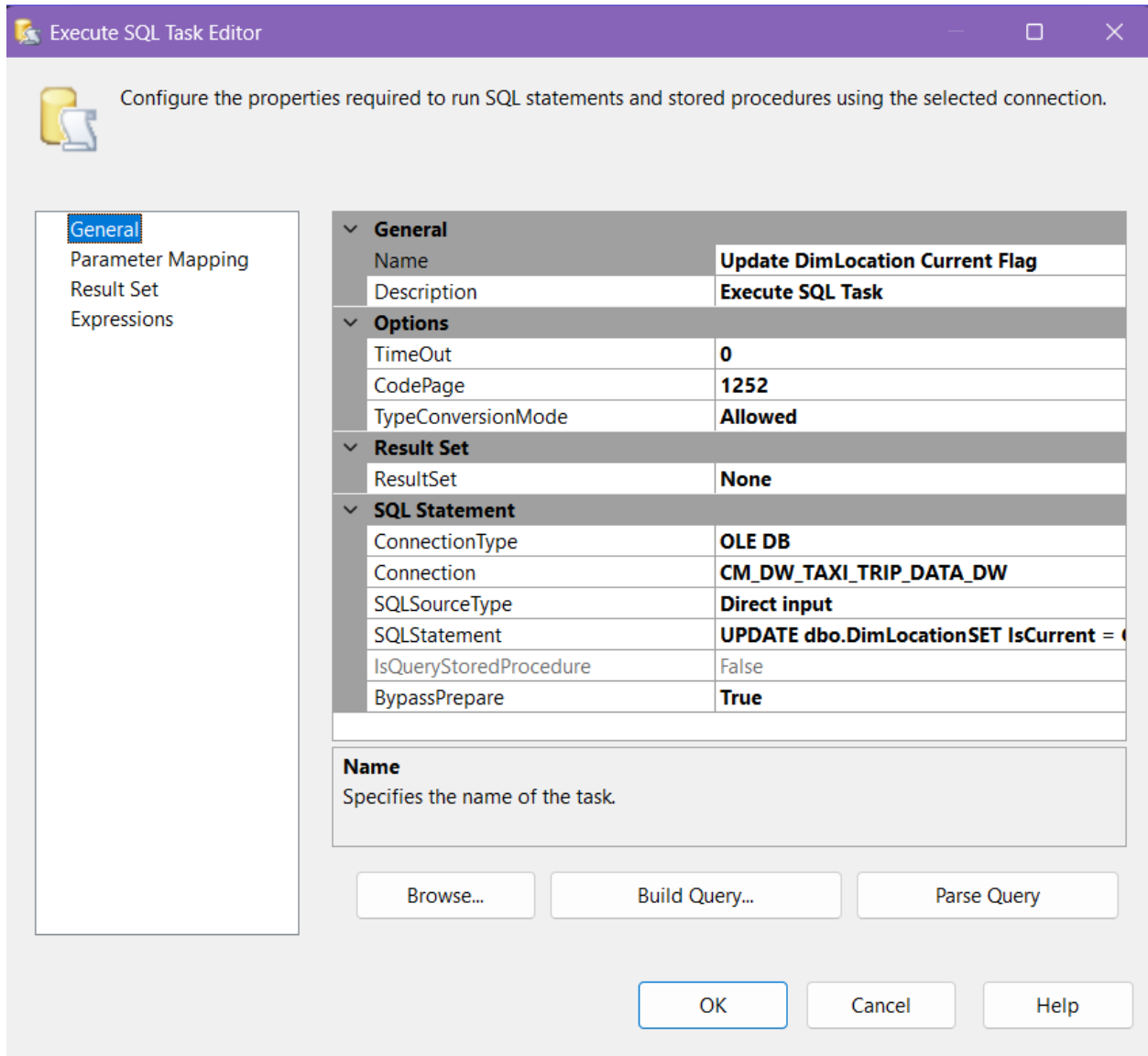
- Slowly Changing Dimension Transformation
 - New Records
 - New Record Output
 - Other Outputs
 - Historical Attribute Output

Help < Back Next > Finish Cancel

5.11. Update DimLocation Current Flag

In this step:

- rows with EndDate IS NULL were marked as current
- rows with a historical end date were marked as non-current



Execute SQL Task Editor

Configure the properties required to run SQL statements and stored procedures using the selected connection.

General

Parameter Mapping
Result Set
Expressions

General	
Name	Update DimLocation Current Flag
Description	Execute SQL Task
Options	
TimeOut	0
CodePage	1252
TypeConversionMode	Allowed
Result Set	
ResultSet	None
SQL Statement	
ConnectionType	OLE DB
Connection	CM_DW_TAXI_TRIP_DATA_DW
SQLSourceType	Direct input
SQLStatement	UPDATE dbo.DimLocationSET IsCurrent = 1
IsQueryStoredProcedure	False
BypassPrepare	True

Name
Specifies the name of the task.

Browse... Build Query... Parse Query

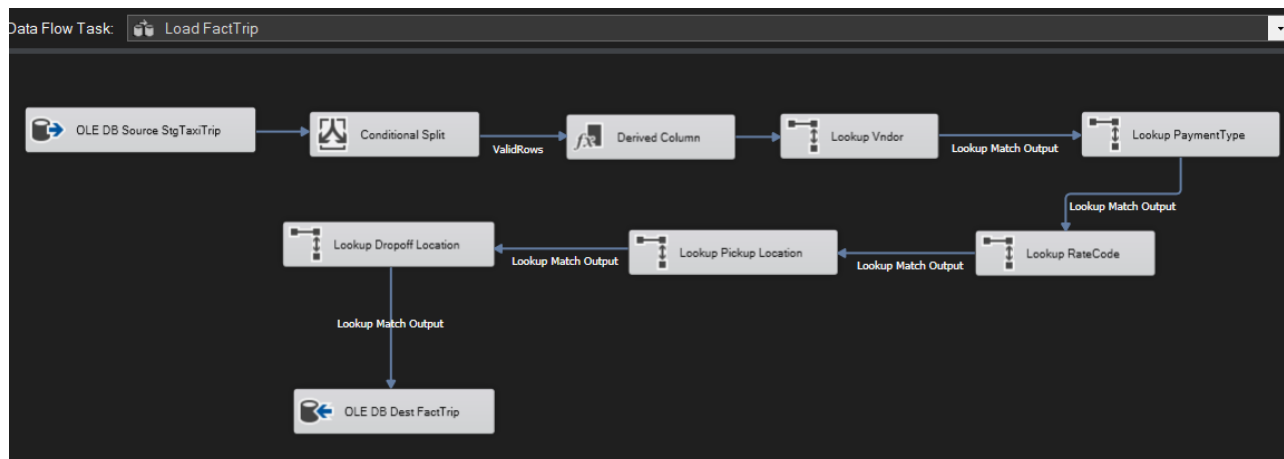
OK Cancel Help

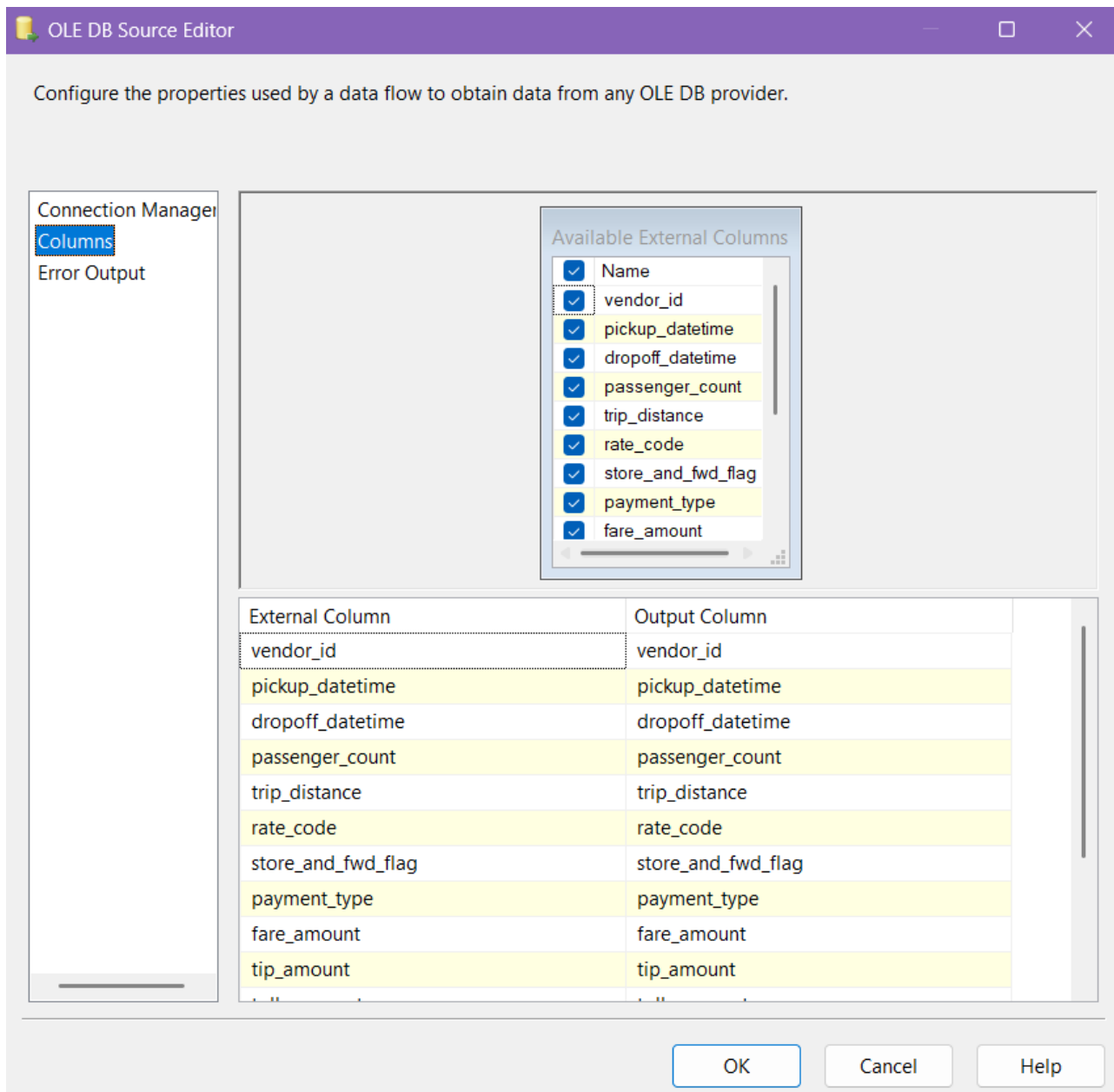
5.12. Load FactTrip

Valid trip records were transformed, matched with surrogate keys from the dimensions, and loaded into FactTrip.

In this step:

- trip data was read from StgTaxiTrip
- a Conditional Split was used to keep only valid records
- a Derived Column transformation was used to create PickupDateKey, DropoffDateKey, TripDurationMinutes, and StoreAndFwdFlag
- Lookup transformations were used to retrieve:
 - VendorKey
 - PaymentTypeKey
 - RateCodeKey
 - PickupLocationKey
 - DropoffLocationKey
- The transformed records were loaded into FactTrip





Derived Column Transformation Editor

Specify the expressions used to create new column values, and indicate whether the values update existing columns or populate new columns.

Variables and Parameters

Columns

Mathematical Functions

String Functions

Date/Time Functions

NULL Functions

Type Casts

Operators

Description:

Derived Column Name	Derived Column	Expression	Data Type	Location
PickupDateKey	<add as new column>	(DT_I4)(YEAR((DT_DBTIMESTAMP)pickup_date...	four-byte signed inte...	
DropoffDateKey	<add as new column>	(DT_I4)(YEAR((DT_DBTIMESTAMP)dropoff_dat...	four-byte signed inte...	
TripDurationMinutes	<add as new column>	DATEDIFF("minute",(DT_DBTIMESTAMP)pickup...	four-byte signed inte...	
StoreAndFwdFlag	<add as new column>	(DT_STR,1,1252)store_and_fwd_flag	string [DT_STR]	1

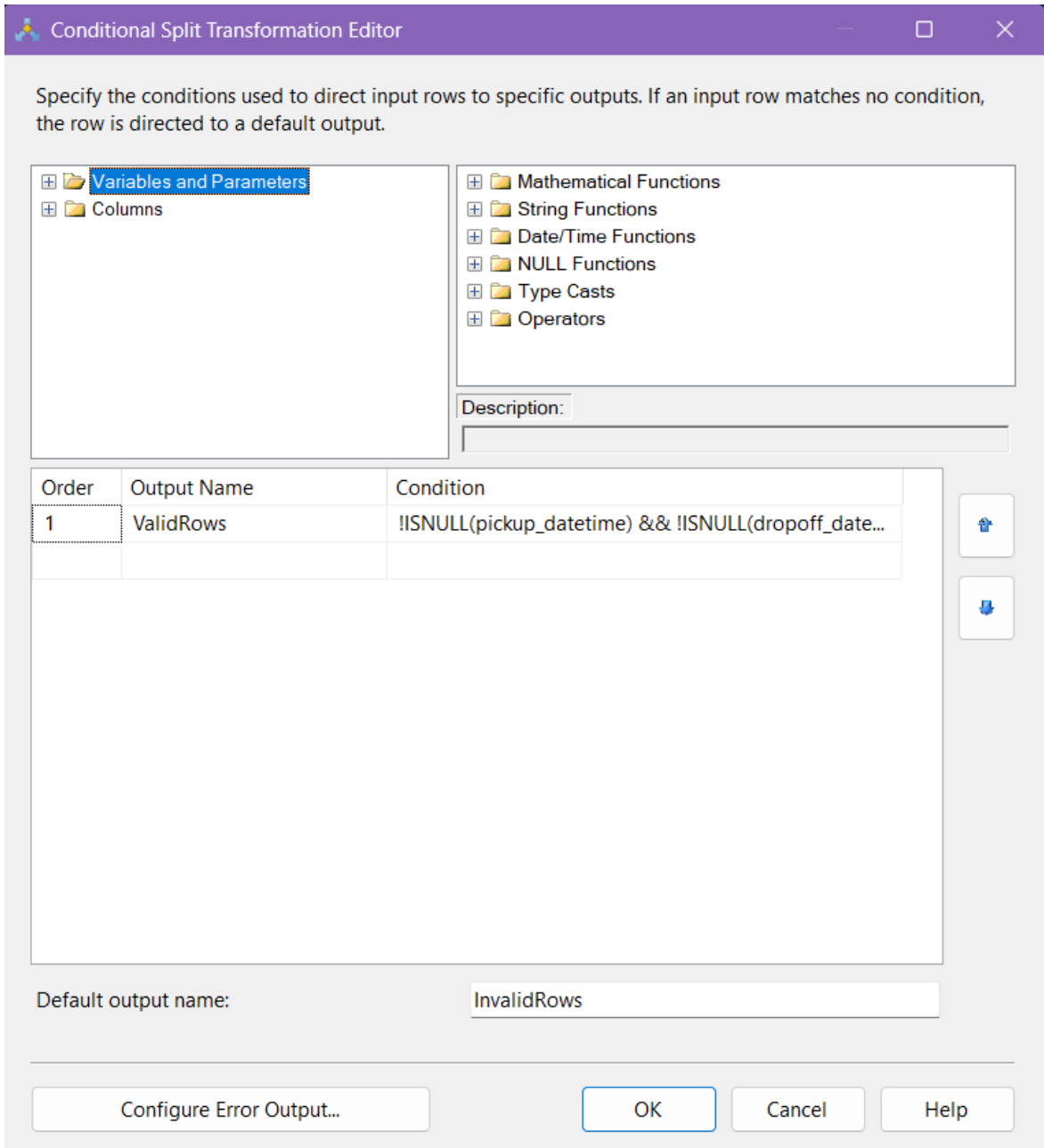
Configure Error Output...

OK

Cancel

Help

35



Lookup Transformation Editor

This transform enables the performance of simple equi-joins between the input and a reference data set.

General

Connection

Columns

Advanced

Error Output

Available Input Col...

Name

vendor_id

pickup_datetime

dropoff_datetime

passenger_count

trip_distance

rate_code

store_and_fwd_...

payment_type

fare_amount

Available Lookup Columns

☒

Name

Index

☒

VendorKey

☐

VendorID

☐

VendorName

Lookup Column	Lookup Operation	Output Alias
VendorKey	<add as new column>	VendorKey

OK

Cancel

Help

OLE DB Destination Editor

Configure the properties used to insert data into a relational database using an OLE DB provider.

Connection Manager

Mappings

Error Output

Available Input Col...

Name
vendor_id
pickup_datetime
dropoff_datetime
passenger_count
trip_distance
rate_code
store_and_fwd_...
payment_type
fare_amount

Available Destination...

Name
FactTripKey
TripID
PickupDateKey
DropoffDateKey
PickupLocationKey
DropoffLocationK...
PaymentTypeKey
RateCodeKey
VendorKey

Input Column	Destination Column
<ignore>	FactTripKey
<ignore>	TripID
PickupDateKey	PickupDateKey
DropoffDateKey	DropoffDateKey
PickupLocationKey	PickupLocationKey
DropoffLocationKey	DropoffLocationKey
PaymentTypeKey	PaymentTypeKey
RateCodeKey	RateCodeKey
VendorKey	VendorKey
StoreAndFwdFlag	StoreAndFwdFlag

OK

Cancel

Help

5.13. Update TripID

A generated trip identifier was assigned to each fact table row after loading the fact table.

Execute SQL Task Editor

Configure the properties required to run SQL statements and stored procedures using the selected connection.

General
Parameter Mapping
Result Set
Expressions

General	
Name	Update TripID
Description	Execute SQL Task
Options	
TimeOut	0
CodePage	1252
TypeConversionMode	Allowed
Result Set	
ResultSet	None
SQL Statement	
ConnectionType	OLE DB
Connection	CM_DW_TAXI_TRIP_DATA_DW
SQLSourceType	Direct input
SQLStatement	UPDATE dbo.FactTripSET TripID = 'TRIP_' +
IsQueryStoredProcedure	False
BypassPrepare	True

Name
Specifies the name of the task.

Browse... Build Query... Parse Query

OK Cancel Help

5.14. Execute the Package and Validate the Results

After all tasks were configured, the package was executed successfully. The loaded data was then validated using SQL queries.

```
1  USE TAXI_TRIP_DATA_DW;
2  GO
3
4  SELECT COUNT(*) AS DimDateRows FROM dbo.DimDate;
5  SELECT COUNT(*) AS DimVendorRows FROM dbo.DimVendor;
6  SELECT COUNT(*) AS DimPaymentTypeRows FROM dbo.DimPaymentType;
7  SELECT COUNT(*) AS DimRateCodeRows FROM dbo.DimRateCode;
8  SELECT COUNT(*) AS DimLocationRows FROM dbo.DimLocation;
9  SELECT COUNT(*) AS FactTripRows FROM dbo.FactTrip;
10
11 SELECT TOP 10 * FROM dbo.FactTrip;
12 SELECT TOP 10 * FROM dbo.DimLocation;
```

100 % No issues found

Results	Messages
DimDateRows	
365	
DimVendorRows	
2	
DimPaymentTypeRows	
6	
DimRateCodeRows	
6	
DimLocationRows	
263	
FactTripRows	
1248317	

100 %

No issues found

Ln: 11, Ch: 1

(34 chars, 1 lines)

TABSCRLFUTF-8

ResultsMessages

	FactTripKey	TripID	PickupDateKey	DropoffDateKey	PickupLocationKey	DropoffLocationKey	PaymentTypeKey	RateCodeKey	VendorKey	StoreAndFwdFlag	PassengerCount	TripDistance	FareAmount	TipAmount	Tolls
1	1	TRIP_1	20180329	20180329	481	267	25	27	12	N	1	18.15	70.00	16.16	10.5
2	2	TRIP_2	20180329	20180329	443	494	25	25	12	N	1	4.59	25.00	5.16	0.00
3	3	TRIP_3	20180329	20180329	495	495	25	25	12	N	1	0.30	3.00	0.76	0.00
4	4	TRIP_4	20180329	20180329	495	349	25	25	12	N	2	16.97	49.50	5.61	5.76
5	5	TRIP_5	20180329	20180329	455	349	25	25	12	N	5	14.45	45.50	10.41	5.76
6	6	TRIP_6	20180329	20180329	451	349	25	25	12	N	1	11.60	42.00	14.57	5.76
7	7	TRIP_7	20180329	20180329	458	455	25	25	11	N	1	5.80	24.00	4.95	0.00
8	8	TRIP_8	20180329	20180329	475	481	25	25	12	N	1	3.38	25.00	5.16	0.00
9	9	TRIP_9	20180329	20180329	455	267	25	27	12	N	1	16.98	85.00	15.00	12.5
10	10	TRIP_10	20180329	20180329	443	481	25	25	12	N	1	4.99	22.00	4.76	0.00

100 % ▾

✔ No issues found

Results Messages

	LocationKey	ZoneID	ZoneName	Borough	StartDate	EndDate	IsCurrent
1	267	1	Newark Airport	EWR	2026-03-26	NULL	1
2	268	3	Allerton/Pelham Gardens	Bronx	2026-03-26	NULL	1
3	269	18	Bedford Park	Bronx	2026-03-26	NULL	1
4	270	20	Belmont	Bronx	2026-03-26	NULL	1
5	271	31	Bronx Park	Bronx	2026-03-26	NULL	1
6	272	32	Bronxdale	Bronx	2026-03-26	NULL	1
7	273	46	City Island	Bronx	2026-03-26	NULL	1
8	274	47	Claremont/Bathgate	Bronx	2026-03-26	NULL	1
9	275	51	Co-Op City	Bronx	2026-03-26	NULL	1
10	276	58	Country Club	Bronx	2026-03-26	NULL	1

	VendorKey	VendorID	VendorName
1	11	1	Creative Mobile Technologies, LLC
2	12	2	VeriFone Inc.

100 % ▾

✔ No issues found

Results Messages

	PaymentTypeKey	PaymentID	PaymentName
1	25	1	Credit Card
2	26	2	Cash
3	27	3	No Charge
4	28	4	Dispute
5	29	5	Unknown
6	30	6	Voided Trip

	RateCodeKey	RateCodeID	RateCodeName
1	25	1	Standard Rate
2	26	2	JFK
3	27	3	Newark
4	28	4	Nassau or Westchester
5	29	5	Negotiated Fare
6	30	6	Group Ride

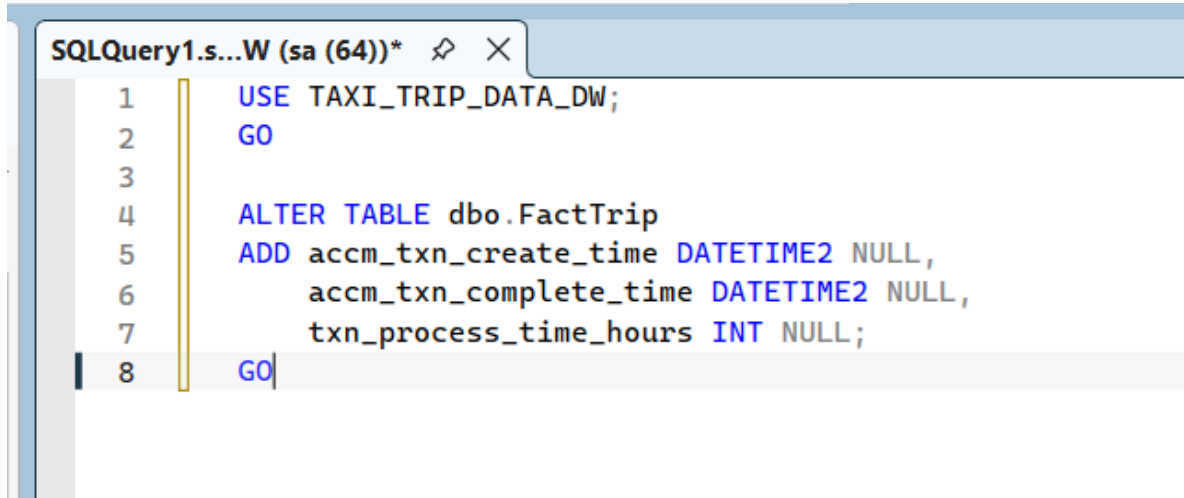
6. ETL development – Accumulating fact tables

The FactTrip table was extended to support an accumulating fact table design. In addition to the existing fact measures, three new attributes were introduced to track the lifecycle of a transaction. These attributes are accm_txn_create_time, accm_txn_complete_time, and txn_process_time_hours.

TripID was used as the fact table natural key for the update process.

6.1.Extension of the Fact Table.

The fact table was extended with three additional columns to support the accumulating fact table design

A screenshot of a SQL query window titled "SQLQuery1.s...W (sa (64))*". The window contains a SQL script with line numbers 1 through 8. The script starts with "USE TAXI_TRIP_DATA_DW;" on line 1, followed by "GO" on line 2. Line 3 is blank. Line 4 starts with "ALTER TABLE dbo.FactTrip". Line 5 has "ADD accm_txn_create_time DATETIME2 NULL,". Line 6 has "accm_txn_complete_time DATETIME2 NULL,". Line 7 has "txn_process_time_hours INT NULL;". Line 8 ends with "GO".

```
1  USE TAXI_TRIP_DATA_DW;
2  GO
3
4  ALTER TABLE dbo.FactTrip
5  ADD accm_txn_create_time DATETIME2 NULL,
6      accm_txn_complete_time DATETIME2 NULL,
7      txn_process_time_hours INT NULL;
8  GO
```

6.2.Updating the Main ETL Package

The existing Main_ETL package was modified so that accm_txn_create_time is populated when the fact table is initially loaded.

In the Load FactTrip Data Flow, a Derived Column transformation was updated to include accm_txn_create_time = GETDATE()

Derived Column Transformation Editor

Specify the expressions used to create new column values, and indicate whether the values update existing columns or populate new columns.

Variables and Parameters
Columns

Mathematical Functions
String Functions
Date/Time Functions
NULL Functions
Type Casts
Operators

Description:

Derived Column Name	Derived Column	Expression	Data Type
PickupDateKey	<add as new column>	(DT_I4)(YEAR((DT_DBTIMESTAMP)pickup_date...	four-byte signed inte...
DropoffDateKey	<add as new column>	(DT_I4)(YEAR((DT_DBTIMESTAMP)dropoff_dat...	four-byte signed inte...
TripDurationMinutes	<add as new column>	DATEDIFF("minute",(DT_DBTIMESTAMP)pickup...	four-byte signed inte...
StoreAndFwdFlag	<add as new column>	(DT_STR,1,1252)store_and_fwd_flag	string [DT_STR]
accm_txn_create_time	<add as new column>	GETDATE()	database timestamp ...

Configure Error Output... OK Cancel Help

OLE DB Destination Editor

Configure the properties used to insert data into a relational database using an OLE DB provider.

Connection Manager
Mappings
Error Output

Available Input Col...

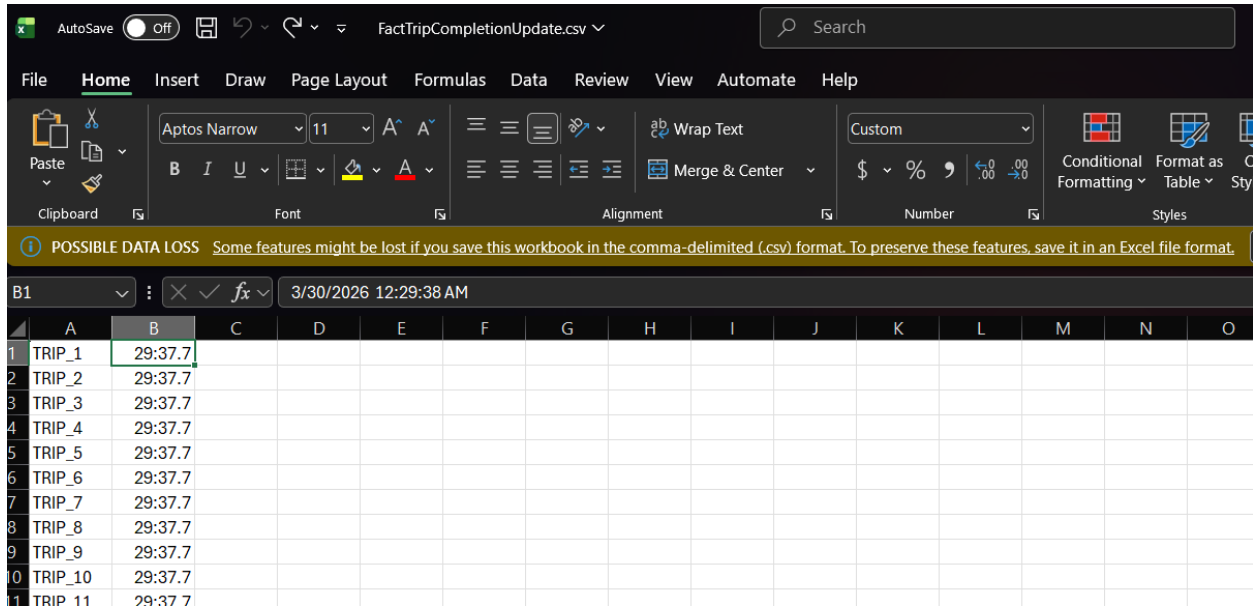
Available Destination...

Input Column	Destination Column
passenger_count	PassengerCount
trip_distance	TripDistance
fare_amount	FareAmount
tip_amount	TipAmount
tolls_amount	TollsAmount
total_amount	TotalAmount
TripDurationMinutes	TripDurationMinutes
accm_txn_create_time	accm_txn_create_time
<ignore>	accm_txn_complete_time
<ignore>	txn_process_time_hours

OK Cancel Help

6.3.Preparation of the Update Dataset

A separated CSV file named FactTripCompletionUpdate.csv was created using the TripID values from the fact table together with generated completion timestamps..



	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	TRIP_1	29:37.7													
2	TRIP_2	29:37.7													
3	TRIP_3	29:37.7													
4	TRIP_4	29:37.7													
5	TRIP_5	29:37.7													
6	TRIP_6	29:37.7													
7	TRIP_7	29:37.7													
8	TRIP_8	29:37.7													
9	TRIP_9	29:37.7													
10	TRIP_10	29:37.7													
11	TRIP_11	29:37.7													

6.4.Creation of the Update Staging Table

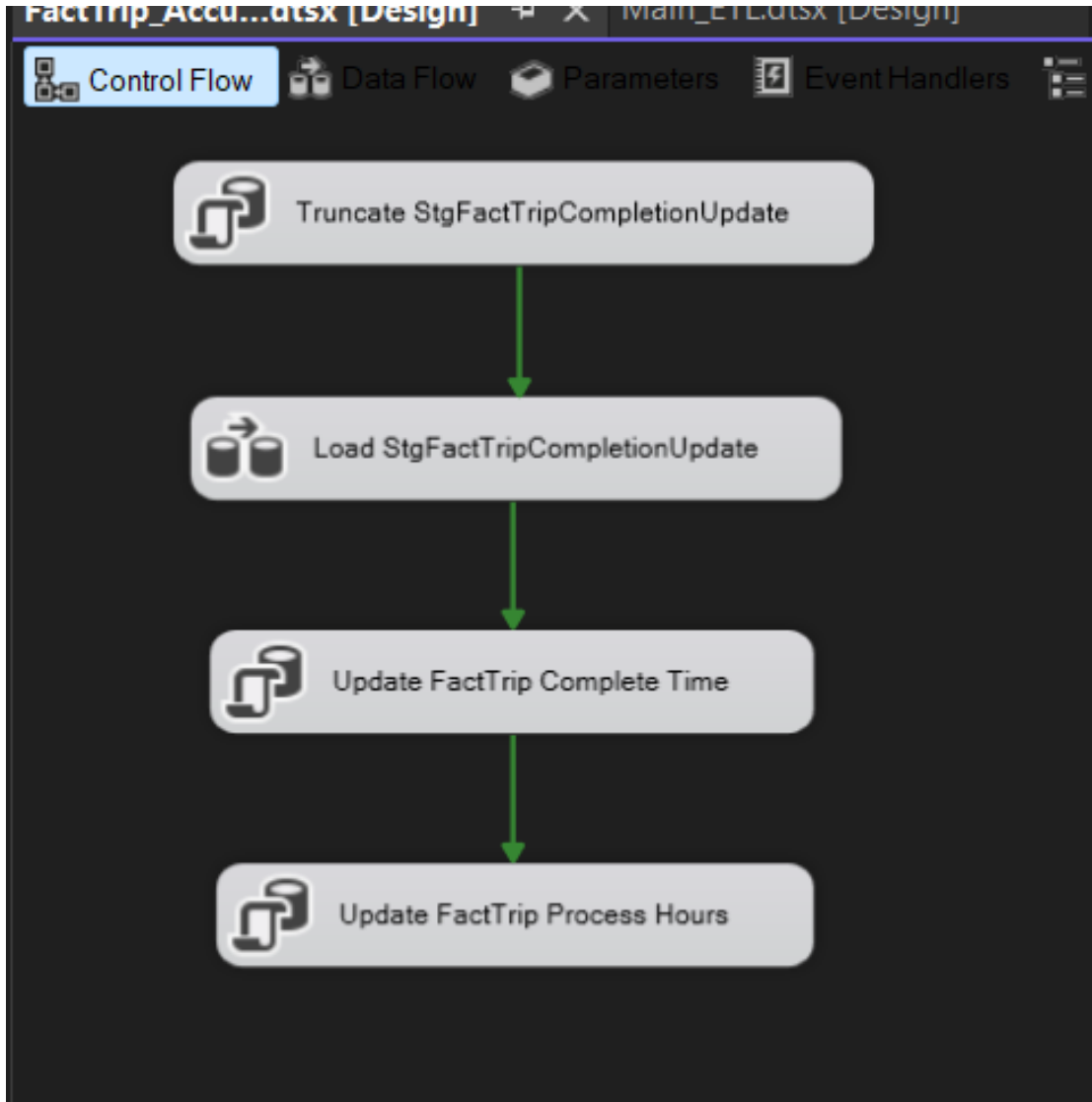
A staging table named StgFactTripCompletionUpdate was created in the staging database to temporarily store the records from the update CSV file.

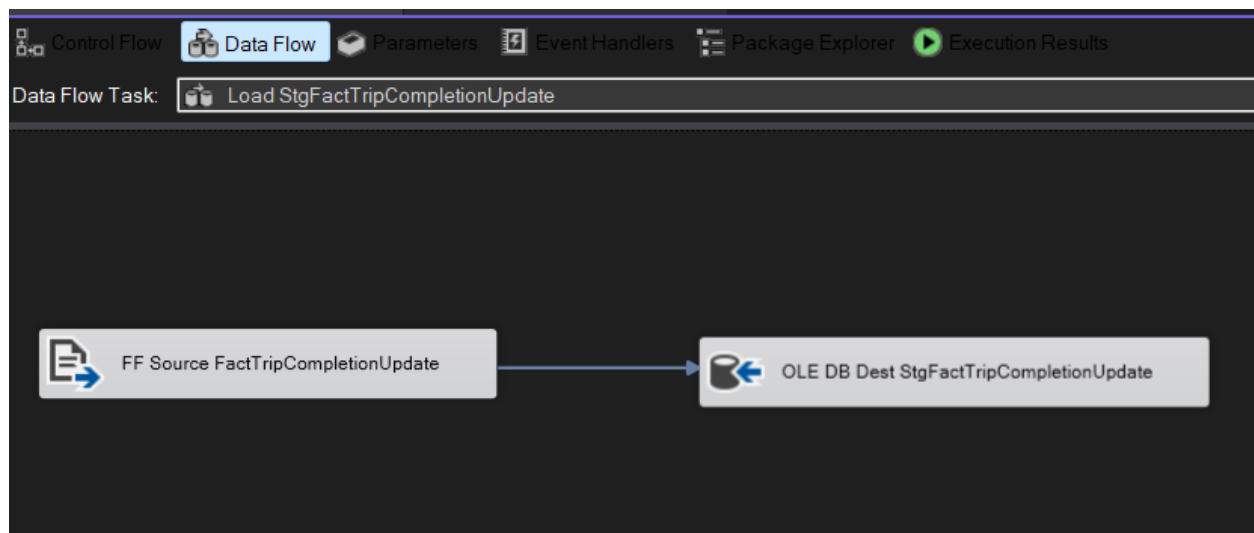
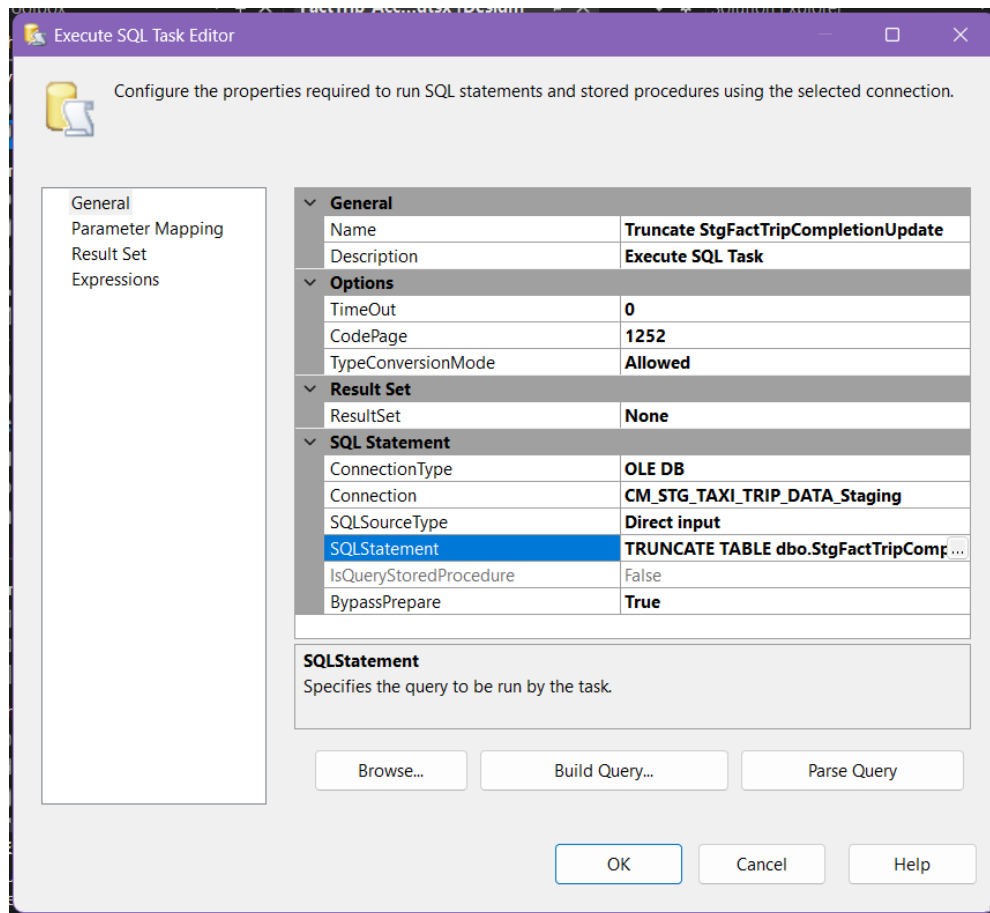
```
QLQuery1.s...ng (sa (92))* ✕ X
1  USE TAXI_TRIP_DATA_Staging;
2  GO
3
4  CREATE TABLE dbo.StgFactTripCompletionUpdate (
5      TripID VARCHAR(100) NOT NULL,
6      accm_txn_complete_time DATETIME2 NOT NULL
7  );
8  GO
```

6.5. Creation of the Separate SSIS Package

A separate SSIS package was created to handle the accumulating fact table update process.

Package name: FactTrip_Accumulating_Update.dtsx





Flat File Source Editor

Configure the properties used to connect to and obtain data from a text file.

Connection Manager
Columns
Error Output

Available External Columns

- ☒ Name
- ☒ TripID
- ☒ accm_txn_complete_time

External Column	Output Column
TripID	TripID
accm_txn_complete_time	accm_txn_complete_time

OLE DB Destination Editor

Configure the properties used to insert data into a relational database using an OLE DB provider.

Connection Manager
Mappings
Error Output

Available Input Columns

- ☐ Name
- ☐ TripID
- ☒ accm_txn_complete_time

Available Destination Columns

- ☐ Name
- ☐ TripID
- ☒ accm_txn_complete_time

Input Column	Destination Column
TripID	TripID
accm_txn_complete_time	accm_txn_complete_time

OK Cancel Help

Execute SQL Task Editor

Configure the properties required to run SQL statements and stored procedures using the selected connection.

General
Parameter Mapping
Result Set
Expressions

General	
Name	Update FactTrip Complete Time
Description	Execute SQL Task
Options	
TimeOut	0
CodePage	1252
TypeConversionMode	Allowed
Result Set	
ResultSet	None
SQL Statement	
ConnectionType	OLE DB
Connection	CM_DW_TAXI_TRIP_DATA_DW
SQLSourceType	Direct input
SQLStatement	UPDATE fSET f.accm_txn_complete_time ...
IsQueryStoredProcedure	False
BypassPrepare	True

SQLStatement
Specifies the query to be run by the task.

Browse... Build Query... Parse Query

OK Cancel Help

Execute SQL Task Editor

Configure the properties required to run SQL statements and stored procedures using the selected connection.

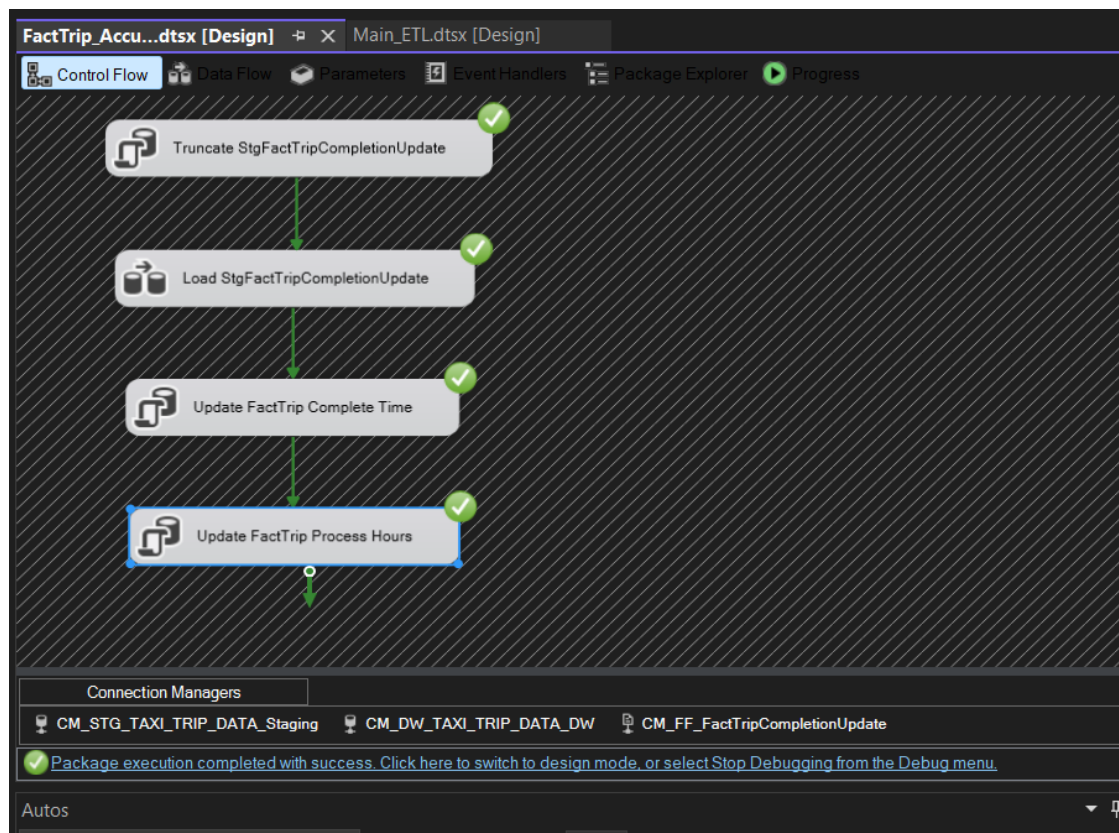
General
Parameter Mapping
Result Set
Expressions

General	
Name	Update FactTrip Process Hours
Description	Execute SQL Task
Options	
TimeOut	0
CodePage	1252
TypeConversionMode	Allowed
Result Set	
ResultSet	None
SQL Statement	
ConnectionType	OLE DB
Connection	CM_DW_TAXI_TRIP_DATA_DW
SQLSourceType	Direct input
SQLStatement	UPDATE dbo.FactTripSET txn_process_ti ...
IsQueryStoredProcedure	False
BypassPrepare	True

SQLStatement
Specifies the query to be run by the task.

Browse... Build Query... Parse Query

OK Cancel Help



SQLQuery1.s...W (sa (92))*

```

37  USE TAXI_TRIP_DATA_DW;
38  GO
39
40  SELECT TOP 20
41      FactTripKey,
42      TripID,
43      accm_txn_create_time,
44      accm_txn_complete_time,
45      txn_process_time_hours
46  FROM dbo.FactTrip
47  ORDER BY FactTripKey;
  
```

100 % 1 0

Results Messages

	FactTripKey	TripID	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours
1	1	TRIP_1	2026-03-27 11:29:37.6766667	2026-03-30 00:29:37.6766667	61
2	2	TRIP_2	2026-03-27 11:29:37.6766667	2026-03-30 01:29:37.6766667	62
3	3	TRIP_3	2026-03-27 11:29:37.6766667	2026-03-30 02:29:37.6766667	63
4	4	TRIP_4	2026-03-27 11:29:37.6766667	2026-03-30 03:29:37.6766667	64
5	5	TRIP_5	2026-03-27 11:29:37.6766667	2026-03-30 04:29:37.6766667	65
6	6	TRIP_6	2026-03-27 11:29:37.6766667	2026-03-30 05:29:37.6766667	66
7	7	TRIP_7	2026-03-27 11:29:37.6766667	2026-03-30 06:29:37.6766667	67
8	8	TRIP_8	2026-03-27 11:29:37.6766667	2026-03-30 07:29:37.6766667	68
9	9	TRIP_9	2026-03-27 11:29:37.6766667	2026-03-30 08:29:37.6766667	69
10	10	TRIP_10	2026-03-27 11:29:37.6766667	2026-03-29 23:29:37.6766667	60
11	11	TRIP_11	2026-03-27 11:29:37.6766667	2026-03-29 22:29:37.6766667	59
12	12	TRIP_12	2026-03-27 11:29:37.6766667	2026-03-29 21:29:37.6766667	58
13	13	TRIP_13	2026-03-27 11:29:37.6766667	2026-03-29 20:29:37.6766667	57
14	14	TRIP_14	2026-03-27 11:29:37.6766667	2026-03-29 19:29:37.6766667	56
15	15	TRIP_15	2026-03-27 11:29:37.6766667	2026-03-29 18:29:37.6766667	55
16	16	TRIP_16	2026-03-27 11:29:37.6766667	2026-03-29 17:29:37.6766667	54
17	17	TRIP_17	2026-03-27 11:29:37.6766667	2026-03-29 16:29:37.6766667	53

Query executed successfully. MSN